ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC: CHUYÊN ĐỀ ASP.NET

HỌC KỲ 7, NĂM HỌC 2026

XÂY DỰNG WEBSITE BÁN SẢN PHẨM CÔNG NGHỆ APPLE (APPLE STORE)

Giảng viên hướng dẫn:
TS. Nguyễn Nhứt Lam

Sinh viên thực hiện:
Họ tên: Lê Bình
MSSV: 470124170
Lớp: VX24TTK7

Vĩnh Long, tháng 10 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày ..... tháng ..... năm 2026

Thành viên hội đồng

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy TS. Nguyễn Nhứt Lam, giảng viên hướng dẫn môn Chuyên đề ASP.NET, đã tận tình hướng dẫn, góp ý và định hướng cho em trong suốt quá trình thực hiện đồ án này. Những kiến thức về ASP.NET Core mà thầy truyền đạt là nền tảng để em xây dựng được website Apple Store.

Em cũng xin cảm ơn Ban Giám hiệu, Trường Kỹ thuật và Công nghệ, Khoa Công nghệ Thông tin cùng các thầy cô đã tạo điều kiện học tập và nghiên cứu để em hoàn thành đồ án.

Em xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ em trong thời gian học tập và thực hiện đề tài.

Do thời gian và kinh nghiệm còn hạn chế, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được ý kiến đóng góp của thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Sinh viên thực hiện

Lê Bình

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU....1
1.1. Lý do chọn đề tài....1
1.2. Mục tiêu của đề tài....1
1.2.1. Mục tiêu chung....1
1.2.2. Mục tiêu cụ thể....1
1.3. Đối tượng và phạm vi nghiên cứu....2
1.3.1. Đối tượng nghiên cứu....2
1.3.2. Phạm vi nghiên cứu....2
1.4. Phương pháp thực hiện....2
1.5. Bố cục báo cáo....3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT....4
2.1. Tổng quan về website thương mại điện tử bán sản phẩm công nghệ....4
2.2. Nền tảng .NET và ASP.NET Core MVC....4
2.2.1. Giới thiệu .NET và ASP.NET Core....4
2.2.2. Mô hình kiến trúc MVC....4
2.2.3. Razor, Tag Helper, Partial View và View Component....4
2.2.4. Areas....5
2.2.5. Dependency Injection và Options pattern....5
2.2.6. Model binding và model validation....5
2.2.7. Chống giả mạo yêu cầu (anti-forgery)....6
2.3. Entity Framework Core....6
2.3.1. ORM và cách tiếp cận Code First....6
2.3.2. Cấu hình bằng IEntityTypeConfiguration....6
2.3.3. Migration....6
2.3.4. Giao dịch và cập nhật có điều kiện....6
2.4. ASP.NET Core Identity....7
2.5. Hệ quản trị cơ sở dữ liệu....7
2.6. Các thư viện và công nghệ hỗ trợ....8
2.7. Kiểm thử phần mềm....8
2.7.1. Kiểm thử tự động với xUnit....8
2.7.2. Phát triển hướng kiểm thử (TDD) và kiểm thử đột biến....8
2.7.3. Kiểm thử trên trình duyệt thật với Playwright....9
2.8. Công cụ phát triển và quản lý mã nguồn....9

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....10
3.1. Phân tích yêu cầu hệ thống.....10
3.1.1. Tác nhân.....10
3.1.2. Yêu cầu chức năng.....10
3.1.3. Yêu cầu phi chức năng.....11
3.2. Đặc tả các trường hợp sử dụng.....12
3.2.1. Danh sách use case và trạng thái.....12
3.2.2. Đặc tả use case Đăng ký tài khoản (use case 1, 2).....14
3.2.3. Đặc tả use case Đặt hàng và áp dụng voucher (use case 15, 16).....15
3.2.4. Đặc tả use case Thanh toán trực tuyến (use case 17).....16
3.2.5. Đặc tả use case Xử lý đơn hàng (use case 22, 23, 24).....17
3.2.6. Đặc tả use case Nhập hàng vào kho (use case 20).....18
3.2.7. Đặc tả use case Bán hàng tại quầy (use case 21).....19
3.3. Thiết kế cơ sở dữ liệu.....19
3.3.1. Tổng quan.....19
3.3.2. Các bảng chính.....20
3.3.3. Mối quan hệ giữa các bảng.....22
3.3.4. Một số điều chỉnh so với thiết kế gốc.....23
3.4. Thiết kế kiến trúc.....23
3.5. Thiết kế các quy tắc tính tiền.....24
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM.....25
4.1. Môi trường phát triển.....25
4.2. Cấu trúc mã nguồn.....25
4.2.1. Một số module chính (kèm mã minh họa).....26
4.3. Giao diện phía khách hàng.....30
4.3.1. Trang chủ và danh mục sản phẩm.....30
4.3.2. Tìm kiếm, lọc theo giá và sắp xếp.....32
4.3.3. Trang cấu hình sản phẩm.....33
4.3.4. Đăng ký, xác thực OTP và đăng nhập.....34
4.3.5. So sánh sản phẩm.....36
4.3.6. Giỏ hàng, thanh toán và đơn hàng.....37
4.3.7. Đánh giá sản phẩm.....39
4.4. Giao diện trang quản trị.....40
4.4.1. Tổng quan và xử lý đơn hàng.....41
4.4.2. Nhập kho và tồn kho.....42

4.4.3. Bán hàng tại quầy và in hóa đơn hàng loạt.....44
4.4.4. Sản phẩm, voucher, khuyến mãi và giá.....47
4.4.5. Báo cáo, đánh giá và tài khoản.....50
4.5. Kiểm thử và đánh giá.....51
4.5.1. Kiểm thử tự động.....51
4.5.2. Kiểm thử trên trình duyệt.....51
4.5.3. Kiểm thử đột biến và rà soát mã.....52
4.5.4. Một số lỗi phát hiện và đã khắc phục.....52
4.5.5. Quy trình quản lý mã nguồn.....53
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....54
5.1. Kết luận.....54
5.1.1. Kết quả đạt được.....54
5.1.2. Ý nghĩa của đề tài.....54
5.1.3. Hạn chế.....54
5.2. Hướng phát triển.....55
TÀI LIỆU THAM KHẢO.....56

DANH MỤC HÌNH

Hình 3.1: Sơ đồ use case theo tác nhân.....14
Hình 3.2: Lưu đồ đặt hàng và áp dụng voucher.....16
Hình 3.3: Lưu đồ trạng thái đơn hàng (OrderTransitions).....18
Hình 3.4: ERD: Danh mục và sản phẩm.....20
Hình 3.5: ERD: Tài khoản và mua hàng.....20
Hình 3.6: ERD: Tương tác, giá và kho.....20
Hình 4.1: WholeDongAttribute, thuộc tính kiểm tra số tiền nguyên đồng.....27
Hình 4.2: OrderTransitions, bảng quy tắc chuyển trạng thái đơn hàng.....28
Hình 4.3: PriceService, đổi giá hàng loạt trong một giao dịch (rút gọn).....29
Hình 4.4: UserStore, Identity đọc ghi trên bảng Users của đặc tả.....30
Hình 4.5: Trang chủ.....31
Hình 4.6: Trang danh mục iPhone (khung đỏ đánh dấu khối chính).....31
Hình 4.7: Dải mẫu Apple Watch: hai mẫu chưa có ảnh hiện hình vẽ đồng hồ (khung đỏ đánh dấu khối chính).....32
Hình 4.8: Tìm kiếm kết hợp lọc theo khoảng giá và sắp xếp.....32
Hình 4.9: Trang cấu hình sản phẩm có khuyến mãi (khung đỏ đánh dấu khối chính)..33
Hình 4.10: Trang đăng ký tài khoản (khung đỏ đánh dấu khối chính).....34
Hình 4.11: Trang nhập mã OTP gửi qua email.....35
Hình 4.12: Trang đăng nhập (khung đỏ đánh dấu khối chính).....36
Hình 4.13: Trang so sánh sản phẩm (khung đỏ đánh dấu khối chính).....36
Hình 4.14: Giỏ hàng (khung đỏ đánh dấu khối chính).....37
Hình 4.15: Trang thanh toán đã áp dụng voucher (khung đỏ đánh dấu khối chính).....37
Hình 4.16: Trang đơn hàng sau khi đặt (khung đỏ đánh dấu khối chính).....38
Hình 4.17: Danh sách đơn hàng của khách (khung đỏ đánh dấu khối chính).....38
Hình 4.18: Tra cứu đơn hàng bằng mã đơn và số điện thoại (khung đỏ đánh dấu khối chính).....39
Hình 4.19: Khối đánh giá trên trang sản phẩm (khung đỏ đánh dấu khối chính).....40
Hình 4.20: Trang tổng quan của quản trị viên (khung đỏ đánh dấu khối chính).....41
Hình 4.21: Danh sách đơn hàng theo trạng thái (khung đỏ đánh dấu khối chính).....41
Hình 4.22: Chi tiết đơn hàng và các nút xử lý theo trạng thái (khung đỏ đánh dấu khối chính).....42
Hình 4.23: Lập phiếu nhập kho (khung đỏ đánh dấu khối chính).....43

Hình 4.24: Phiếu nhập đã lưu: tồn trước và tồn sau của từng dòng (khung đỏ đánh dấu khối chính).....43
Hình 4.25: Danh sách tồn kho (khung đỏ đánh dấu khối chính).....44
Hình 4.26: Bán hàng tại quầy: bảng tính tiền trước khi hoàn tất.....45
Hình 4.27: Đơn bán tại quầy: kênh Tại quầy và nhân viên bán (khung đỏ đánh dấu khối chính).....46
Hình 4.28: In nhiều hóa đơn cùng lúc.....46
Hình 4.29: Quản lý sản phẩm (khung đỏ đánh dấu khối chính).....47
Hình 4.30: Quản lý voucher (khung đỏ đánh dấu khối chính).....48
Hình 4.31: Quản lý khuyến mãi (khung đỏ đánh dấu khối chính).....48
Hình 4.32: Đổi giá hàng loạt và lịch sử đổi giá (khung đỏ đánh dấu khối chính).....49
Hình 4.33: Báo cáo doanh thu, xuất Excel và PDF (khung đỏ đánh dấu khối chính)...50
Hình 4.34: Quản lý đánh giá: trả lời và ẩn (khung đỏ đánh dấu khối chính).....50
Hình 4.35: Quản lý tài khoản và vai trò (khung đỏ đánh dấu khối chính).....51

DANH MỤC BẢNG

Bảng 2.1: Thư viện và công nghệ sử dụng.....8
Bảng 3.1: Các tác nhân của hệ thống.....10
Bảng 3.2: Quy tắc nghiệp vụ chính và cách hiện thực.....10
Bảng 3.3: Yêu cầu phi chức năng.....11
Bảng 3.4: Danh sách 37 use case.....12
Bảng 3.5: Quy tắc chuyển trạng thái đơn hàng (OrderTransitions).....17
Bảng 3.6: Bảng Products và ProductVariants.....20
Bảng 3.7: Bảng đơn hàng và thanh toán.....21
Bảng 3.8: Các bảng thêm so với đặc tả.....21
Bảng 3.9: Các dự án trong giải pháp.....23
Bảng 4.1: Phần mềm và công cụ.....25
Bảng 4.2: Các thành phần chính trong mã nguồn.....25
Bảng 4.3: Các kịch bản kiểm tra trên trình duyệt (chạy lại ngày 12/10/2026).....51
Bảng 4.4: Một số lỗi phát hiện trong quá trình kiểm thử.....52

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
MVC	Model, View, Controller: mô hình kiến trúc của ASP.NET Core MVC
EF Core	Entity Framework Core: thư viện ánh xạ đối tượng và cơ sở dữ liệu (ORM)
ORM	Object Relational Mapping: ánh xạ đối tượng sang bảng dữ liệu
CSDL	Cơ sở dữ liệu
OTP	One-Time Password: mã xác thực dùng một lần
COD	Cash On Delivery: thanh toán khi nhận hàng
CRUD	Create, Read, Update, Delete: thêm, xem, sửa, xóa
DI	Dependency Injection: tiêm phụ thuộc
CSRF	Cross-Site Request Forgery: giả mạo yêu cầu từ trang khác
HMAC	Hash-based Message Authentication Code: mã xác thực thông điệp dùng hàm băm
SKU	Stock Keeping Unit: mã quản lý từng biến thể hàng hóa
TDD	Test-Driven Development: phát triển hướng kiểm thử, viết test trước
PR	Pull Request: yêu cầu gộp mã trên GitHub
UTC	Coordinated Universal Time: giờ quốc tế
VAT	Value Added Tax: thuế giá trị gia tăng
VNĐ	Việt Nam đồng

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Các sản phẩm công nghệ của Apple như iPhone, iPad, Mac, Apple Watch và AirPods có nhiều phiên bản khác nhau về dung lượng, màu sắc và thị trường phát hành. Người mua thường phải so sánh giữa nhiều cửa hàng và sàn thương mại điện tử, trong khi thông tin về sự khác nhau giữa các biến thể (màu, dung lượng, khu vực) thường không rõ ràng.

Đặc tả yêu cầu mà đề tài dựa vào (đồ án môn Công nghệ phần mềm "Xây dựng website Apple Store", Khoa Công nghệ Thông tin, Trường Đại học Trà Vinh, 2025) nêu ra bốn vấn đề của cách bán hàng hiện tại:

- Thông tin sản phẩm phân tán trên nhiều kênh, khó phân biệt các biến thể.
- Quản lý giỏ hàng và đơn hàng bất tiện, hay sai sót khi đổi số lượng hoặc khi thanh toán khi nhận hàng.
- Quản lý tồn kho và báo cáo còn làm thủ công bằng bảng tính nên dễ nhầm lẫn.
- Tích hợp thanh toán trực tuyến chưa ổn định: một lần cổng thanh toán VNPay hoặc MoMo gọi lại thất bại có thể làm mất đơn hàng.

Từ đặc tả đó, đề tài xây dựng lại hệ thống trên nền tảng ASP.NET Core MVC, là nội dung chính của môn Chuyên đề ASP.NET. Đề tài vừa giải quyết bài toán bán hàng thực tế, vừa là dịp để vận dụng các cơ chế có sẵn của ASP.NET Core như MVC, Razor, Entity Framework Core, Identity và Areas vào một ứng dụng hoàn chỉnh.

1.2. Mục tiêu của đề tài

1.2.1. Mục tiêu chung

Xây dựng một website bán sản phẩm công nghệ Apple chạy được thật, cho phép khách hàng tìm, so sánh, đặt mua và thanh toán sản phẩm, đồng thời cho phép nhân viên và quản trị viên quản lý đơn hàng, sản phẩm, giá, khuyến mãi và báo cáo doanh thu.

1.2.2. Mục tiêu cụ thể

- Hiện thực các trường hợp sử dụng (use case) trong đặc tả: tài khoản và OTP, danh mục sản phẩm, giỏ hàng, đặt hàng, voucher, thanh toán, xử lý đơn hàng, nhập kho, bán hàng tại quầy, quản trị sản phẩm, khuyến mãi, đánh giá và báo cáo.

- Sử dụng đúng các cơ chế có sẵn của ASP.NET Core: mô hình MVC với Razor view, EF Core Code First và migration, ASP.NET Core Identity cho đăng nhập và phân quyền, Area cho trang quản trị, model validation và chống giả mạo yêu cầu (anti-forgery).
- Bảo đảm dữ liệu đúng khi có nhiều người thao tác cùng lúc: không bán quá số lượng tồn kho, không dùng voucher quá số lượt, không ghi đè thay đổi của người khác.
- Kiểm thử kỹ: viết test tự động trước khi viết mã (TDD) và kiểm tra lại toàn bộ trên trình duyệt thật bằng Playwright.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

- Nền tảng ASP.NET Core MVC (.NET 10) và Entity Framework Core.
- Nghiệp vụ bán lẻ sản phẩm công nghệ trực tuyến: danh mục, biến thể, giỏ hàng, đơn hàng, thanh toán, voucher, khuyến mãi, đánh giá, báo cáo.
- Bốn nhóm người dùng: khách vãng lai, khách hàng, nhân viên và quản trị viên.

1.3.2. Phạm vi nghiên cứu

- Website gồm phần cửa hàng cho khách và phần quản trị (khu vực /Admin) cho nhân viên và quản trị viên.
- Đặc tả có 37 use case và đồ án hoàn thành đủ 37, kể cả hai use case của nhân viên cửa hàng là nhập hàng vào kho (use case 20) và bán hàng tại quầy (use case 21).
- Thanh toán VNPay và MoMo được giả lập ngay trong ứng dụng vì không có tài khoản sandbox; chữ ký số của cổng giả lập làm theo cách VNPay ký (HMAC-SHA512).
- Cơ sở dữ liệu khi phát triển là SQLite, nhưng các kiểu dữ liệu được khai báo theo đúng thiết kế cho SQL Server trong đặc tả.

1.4. Phương pháp thực hiện

Đồ án được làm theo từng tính năng nhỏ, mỗi tính năng một nhánh Git và một pull request vào nhánh dev. Với mỗi hành vi có quyết định nghiệp vụ, test được viết trước và chạy để thấy thất bại (RED), sau đó mới viết mã để test qua (GREEN). Sau mỗi tính năng, ứng dụng được chạy thật và kiểm tra trên trình duyệt bằng các kịch bản Playwright, ở cả kích thước màn hình máy tính và điện thoại. Mỗi pull request còn được rà soát lại

theo cách của một lập trình viên có kinh nghiệm, tìm các tình huống chạy song song có thể làm sai dữ liệu.

1.5. Bố cục báo cáo

- Chương 1: Mở đầu, nêu lý do, mục tiêu, phạm vi và phương pháp.
- Chương 2: Cơ sở lý thuyết, trình bày các công nghệ ASP.NET Core được dùng và cách chúng xuất hiện trong mã nguồn của đồ án.
- Chương 3: Phân tích và thiết kế hệ thống: tác nhân, use case, cơ sở dữ liệu, kiến trúc và các luồng nghiệp vụ chính.
- Chương 4: Kết quả thực nghiệm: môi trường, giao diện, kiểm thử và đánh giá.
- Chương 5: Kết luận và hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về website thương mại điện tử bán sản phẩm công nghệ

Một website thương mại điện tử bán sản phẩm công nghệ cần giải quyết các việc chính: trình bày danh mục và biến thể sản phẩm, cho khách chọn và đặt mua, nhận thanh toán, theo dõi đơn hàng và cho cửa hàng quản lý hàng hóa, giá, khuyến mãi và doanh thu. Với sản phẩm Apple, một mẫu máy (ví dụ iPhone 18 Pro Max) có nhiều cấu hình dung lượng, mỗi cấu hình lại có nhiều màu và nhiều khu vực phát hành với giá khác nhau. Vì vậy dữ liệu phải tách rõ sản phẩm, cấu hình và từng biến thể có mã SKU, giá và tồn kho riêng.

2.2. Nền tảng .NET và ASP.NET Core MVC

2.2.1. Giới thiệu .NET và ASP.NET Core

.NET là nền tảng phát triển mã nguồn mở của Microsoft, chạy được trên Windows, macOS và Linux. Đồ án dùng .NET 10, phiên bản hỗ trợ dài hạn (LTS). ASP.NET Core là bộ thư viện web của .NET, dùng để xây dựng website và API. Toàn bộ đồ án được phát triển và chạy trên macOS, điều mà ASP.NET Core hỗ trợ đầy đủ.

2.2.2. Mô hình kiến trúc MVC

MVC chia ứng dụng thành ba phần. **Model** là dữ liệu và quy tắc nghiệp vụ; **View** là phần giao diện, viết bằng Razor; **Controller** nhận yêu cầu từ trình duyệt, gọi phần nghiệp vụ rồi chọn view để trả về. Trong đồ án, ví dụ ProductsController nhận đường dẫn /Products/{slug}, gọi ProductCatalogService để lấy sản phẩm, rồi trả về view Products/Details.cshtml. Nhờ tách như vậy, quy tắc nghiệp vụ nằm trong các lớp dịch vụ và được kiểm thử riêng, còn controller chỉ điều phối.

Đồ án dùng attribute routing (khai báo đường dẫn ngay trên controller, ví dụ [Route("Products")]) để có địa chỉ gọn như /Products/iphone-17 thay vì /Products/Details?slug=iphone-17.

2.2.3. Razor, Tag Helper, Partial View và View Component

Razor là cú pháp trộn HTML với C# trong file .cshtml. Tag Helper là các thuộc tính đặc biệt trong thẻ HTML, ví dụ asp-controller, asp-action để sinh đường dẫn đúng, hay asp-for để gắn ô nhập với thuộc tính của model. Partial view là đoạn giao diện dùng lại ở nhiều trang, ví dụ _ShopCard.cshtml (thẻ sản phẩm) hay _OrderSummary.cshtml

(tóm tắt đơn hàng dùng chung cho khách và nhân viên). View Component là một khối giao diện có logic riêng: CartCountViewComponent hiển thị số món trong giỏ trên thanh điều hướng, ProductReviewsViewComponent hiển thị khối đánh giá trên trang sản phẩm.

2.2.4. Areas

Area là cách ASP.NET Core chia một ứng dụng lớn thành các khu vực có controller và view riêng. Toàn bộ trang quản trị nằm trong Areas/Admin với địa chỉ bắt đầu bằng /Admin. Mỗi controller trong khu vực này được bảo vệ bằng thuộc tính [Authorize(Roles = ...)], ví dụ trang sản phẩm chỉ cho Admin, còn trang đơn hàng, khuyến mãi, giá, kho và bán hàng tại quầy cho cả Admin và Employee.

2.2.5. Dependency Injection và Options pattern

ASP.NET Core có sẵn cơ chế Dependency Injection (DI): các dịch vụ được đăng ký một lần trong Program.cs, ví dụ builder.Services.AddScoped<ICartService, CartService>(), và được tự động truyền vào constructor của controller khi cần. Nhờ DI, khi kiểm thử có thể thay một dịch vụ bằng phiên bản giả (ví dụ dịch vụ gửi email giả) mà không sửa mã.

Options pattern dùng để đọc cấu hình thành một lớp C# có kiểm tra hợp lệ. Đồ án dùng SmtpOptions cho thông tin gửi email và PaymentOptions cho cổng thanh toán, kèm ValidateOnStart để ứng dụng từ chối khởi động nếu cấu hình sai, thay vì lỗi khi người dùng đang thao tác. Mật khẩu email được lưu bằng user-secrets trên máy, không nằm trong mã nguồn.

2.2.6. Model binding và model validation

Model binding tự động chuyển dữ liệu từ form hoặc đường dẫn thành tham số và đối tượng C#. Model validation kiểm tra dữ liệu bằng các thuộc tính như [Required], [StringLength] và kết quả nằm trong ModelState. Đồ án dựa vào ModelState để phát hiện số không đọc được, ví dụ khi nhân viên gõ giá "24.990.000" có dấu chấm, trang sẽ báo lỗi thay vì lưu giá rỗng. Riêng giá trị "1.000" vẫn đọc được thành số 1 (có ba chữ số thập phân), nên đồ án viết thêm một thuộc tính kiểm tra riêng, WholeDongAttribute (kế thừa ValidationAttribute): số tiền phải là số đồng nguyên, không có dấu phân cách. Mọi ô số tiền nhân viên nhập đều dùng chung thuộc tính này: giá sản phẩm, giá biến thể, mức giảm và đơn tối thiểu của voucher, đổi giá theo số tiền và giá nhập kho.

2.2.7. Chống giả mạo yêu cầu (anti-forgery)

Mọi form gửi bằng phương thức POST đều kèm một mã chống giả mạo (anti-forgery token), và action nhận form được đánh dấu [ValidateAntiForgeryToken]. Một trang web lạ không thể lấy được mã này nên không thể lừa trình duyệt của người dùng gửi yêu cầu thay họ (tấn công CSRF). Đồ án có một test riêng (FormTokenTests) đi qua tất cả các trang và kiểm tra từng form POST đều có mã này.

2.3. Entity Framework Core

2.3.1. ORM và cách tiếp cận Code First

Entity Framework Core (EF Core) là thư viện ORM của .NET: các bảng được biểu diễn bằng lớp C# (entity), các truy vấn viết bằng LINQ và EF Core tự sinh câu SQL. Đồ án dùng cách Code First: các lớp entity trong dự án AppleStore.Domain là gốc, cơ sở dữ liệu được tạo ra từ chúng. AppDbContext là lớp đại diện cho phiên làm việc với cơ sở dữ liệu.

2.3.2. Cấu hình bằng IEntityTypeConfiguration

Mỗi bảng có một lớp cấu hình riêng trong Data/Configurations, khai báo khóa chính, khóa ngoại, độ dài chuỗi (HasMaxLength), độ chính xác của tiền (HasPrecision(12, 2) tương ứng decimal(12,2)) và các chỉ mục duy nhất. Nhờ vậy cơ sở dữ liệu giữ đúng hình dạng thiết kế cho SQL Server dù khi phát triển dùng SQLite.

2.3.3. Migration

Migration là các bước thay đổi cấu trúc cơ sở dữ liệu được sinh ra từ mã, lưu trong thư mục Migrations và áp dụng bằng lệnh dotnet ef database update. Đồ án có 24 migration, từ lược đồ ban đầu đến các thay đổi về sau như thêm bảng voucher, cột trả lời đánh giá, loại khuyến mãi, bảng lịch sử đổi giá, bảng phiếu nhập kho, các cột cho đơn bán tại quầy và bảng khóa tra cứu đơn. Dữ liệu mẫu (danh mục, sản phẩm, biến thể, ảnh, voucher mẫu) cũng được đưa vào qua migration nên mọi máy đều có cùng dữ liệu.

2.3.4. Giao dịch và cập nhật có điều kiện

Khi một thao tác cần thay đổi nhiều bảng cùng lúc (ví dụ đặt hàng: trừ tồn kho, tăng lượt dùng voucher, xóa giỏ hàng, tạo đơn), đồ án dùng giao dịch (transaction) của EF Core để tất cả cùng thành công hoặc cùng bị hủy. Để chống hai yêu cầu chạy song song ghi đè nhau, đồ án dùng cập nhật có điều kiện (compare-and-swap) bằng ExecuteUpdateAsync: chỉ trừ tồn kho khi số tồn vẫn đủ, chỉ chuyển trạng thái đơn khi

đơn vẫn ở trạng thái đã thấy, chỉ lưu sửa đổi của quản trị viên khi phiên bản dữ liệu chưa đổi.

2.4. ASP.NET Core Identity

ASP.NET Core Identity là hệ thống tài khoản có sẵn: băm mật khẩu, đăng nhập bằng cookie, khóa tài khoản khi nhập sai nhiều lần, vai trò và quyền. Đồ án dùng UserManager và SignInManager của Identity nhưng giữ nguyên bảng Users theo thiết kế của đặc tả, bằng một UserStore tự viết (Infrastructure/Identity/UserStore.cs) để Identity đọc và ghi vào bảng này. Bốn cột phục vụ Identity được thêm bằng migration: NormalizedEmail, SecurityStamp, AccessFailedCount và LockoutEnd.

- Mật khẩu tối thiểu 8 ký tự; nhập sai 5 lần thì tài khoản bị khóa 5 phút (giá trị mặc định của Identity).
- Vai trò Customer, Employee, Admin được đưa vào cookie đăng nhập và kiểm tra bằng [Authorize(Roles = ...)].
- Security stamp được kiểm tra ở mọi yêu cầu, nên khi quản trị viên đổi vai trò hoặc người dùng đổi mật khẩu, các phiên đăng nhập khác hết hiệu lực ngay.
- Tài khoản quản trị đầu tiên được tạo khi ứng dụng khởi động (AdminSeeder) từ cấu hình trong user-secrets, qua UserManager nên cũng phải theo quy tắc mật khẩu.

2.5. Hệ quản trị cơ sở dữ liệu

Đặc tả thiết kế cơ sở dữ liệu cho SQL Server. Máy phát triển là macOS, không có SQL Server chạy trực tiếp, nên đồ án dùng SQLite khi phát triển: không cần máy chủ, không cần tài khoản và được EF Core hỗ trợ đầy đủ. Các kiểu dữ liệu theo thiết kế SQL Server (nvarchar, decimal(12,2), datetime2) vẫn được khai báo trong cấu hình EF Core, nên khi chuyển sang SQL Server chỉ cần đổi nhà cung cấp và chuỗi kết nối (hướng dẫn trong docs/architecture.md).

Một điểm cần chú ý: SQLite lưu số thập phân dưới dạng chuỗi. Các test của đồ án dùng giá trị như 6.490.000 so với mốc 10.000.000 để chứng minh EF Core vẫn so sánh theo giá trị số chứ không theo thứ tự chuỗi.

2.6. Các thư viện và công nghệ hỗ trợ

Bảng 2.1: Thư viện và công nghệ sử dụng

Thành phần	Vai trò trong đồ án
Bootstrap, jQuery	Bố cục và thành phần giao diện cơ bản đi kèm mẫu dự án ASP.NET Core
CSS và JavaScript riêng (effects.css, effects.js)	Giao diện tối, hiệu ứng hiện dần khi cuộn, chọn màu và khu vực trên trang sản phẩm
GSAP, Lenis, three.js	Hiệu ứng chuyển động và cảnh 3D ở phần đầu trang chủ
MailKit 4.18.1	Gửi email mã OTP và email đơn hàng qua Gmail SMTP
ClosedXML 0.105.1	Xuất báo cáo doanh thu ra file Excel (.xlsx) thật
HMAC-SHA512	Ký và kiểm tra chữ ký của cổng thanh toán giả lập, theo cách VNPay ký
xUnit 2.9.3	Viết và chạy test tự động
Microsoft.AspNetCore.Mvc.Testing 10.0.12	Chạy cả ứng dụng web trong bộ nhớ để test (WebApplicationFactory)
Playwright (playwright-cli)	Điều khiển trình duyệt Chromium thật để kiểm tra giao diện và luồng nghiệp vụ

2.7. Kiểm thử phần mềm

2.7.1. Kiểm thử tự động với xUnit

Test dịch vụ chạy trên cơ sở dữ liệu SQLite trong bộ nhớ với lược đồ thật, nên các ràng buộc như khóa ngoại và chỉ mục duy nhất được kiểm tra thật. Test giao diện dùng WebApplicationFactory để chạy toàn bộ ứng dụng (định tuyến, Identity, cookie, anti-forgery) trong bộ nhớ và gửi yêu cầu HTTP như một trình duyệt. Thời gian được thay bằng đồng hồ cố định (FixedTime dựa trên TimeProvider) để test về hạn voucher, khuyến mãi hay mã OTP không phụ thuộc giờ chạy.

Để kiểm tra tình huống hai yêu cầu chạy cùng lúc, đồ án dùng một interceptor của EF Core (SqlRace) chèn câu SQL của yêu cầu thứ hai vào đúng thời điểm trước khi yêu cầu thứ nhất ghi, rồi kiểm tra dữ liệu cuối cùng vẫn đúng.

2.7.2. Phát triển hướng kiểm thử (TDD) và kiểm thử đột biến

Với mỗi hành vi, test được viết trước và commit riêng ở trạng thái thất bại (RED), sau đó mới commit phần mã làm test qua (GREEN). Sau khi xong, các dòng mã quan

trọng được cố ý làm sai (kiểm thử đột biến) để chắc chắn có test phát hiện; nếu không có test nào thất bại thì phải viết thêm test chặt hơn.

2.7.3. Kiểm thử trên trình duyệt thật với Playwright

Mỗi tính năng có một kịch bản trong thư mục setup/verify-*/. Kịch bản tạo một cơ sở dữ liệu mới bằng chính các migration, khởi động ứng dụng thật, điều khiển trình duyệt Chromium làm như người dùng (đăng ký, nhận OTP, mua hàng, quản trị...), đo giao diện ở chiều rộng 1440 điểm ảnh và 390 điểm ảnh (điện thoại), riêng kịch bản verify-responsive đo mọi trang ở năm chiều rộng 320, 375, 768, 1024 và 1440 điểm ảnh, so sánh dữ liệu với cơ sở dữ liệu và in kết quả đạt hoặc không đạt cho từng mục.

2.8. Công cụ phát triển và quản lý mã nguồn

- dotnet CLI để tạo, build, chạy và test dự án; dotnet-ef (cài như công cụ cục bộ) để tạo và áp dụng migration; dotnet format để kiểm tra định dạng mã.
- Git và GitHub: hai nhánh lâu dài main (phát hành) và dev (tích hợp), mỗi tính năng một nhánh feat/, fix/ hoặc docs/ và gộp vào dev qua pull request.
- Kho mã nguồn: BDucky/ASPNET-VX24TTK7-LeBinh-applestore, tổ chức theo cấu trúc thư mục môn học yêu cầu (setup/, scr/, progress-report/, thesis/).

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1. Tác nhân

Bảng 3.1: Các tác nhân của hệ thống

Tác nhân	Trách nhiệm chính
Khách (Guest)	Đăng ký bằng OTP, đăng nhập, quên mật khẩu, tìm, lọc, xem và so sánh sản phẩm
Khách hàng (Customer)	Mọi việc của khách, cộng thêm: cập nhật hồ sơ, quản lý địa chỉ giao hàng, giỏ hàng, đặt hàng, thanh toán COD hoặc trực tuyến, theo dõi đơn, nhận email, đánh giá sản phẩm đã mua
Nhân viên (Employee)	Kiểm tra tồn kho, nhập hàng và lập phiếu nhập, bán hàng tại quầy, xử lý đơn hàng (xác nhận, giao vận chuyển, cập nhật trạng thái), cập nhật giá và chạy khuyến mãi, trả lời và ẩn đánh giá, xem báo cáo doanh thu, in hóa đơn
Quản trị viên (Admin)	Quản lý sản phẩm, biến thể, ảnh, voucher, khuyến mãi, tài khoản và vai trò, xem báo cáo tổng hợp

3.1.2. Yêu cầu chức năng

Đặc tả nêu các quy tắc nghiệp vụ theo từng biểu mẫu (BM). Bảng 3.2 tóm tắt các quy tắc chính và cách đồ án hiện thực.

Bảng 3.2: Quy tắc nghiệp vụ chính và cách hiện thực

Chức năng	Quy tắc	Hiện thực trong đồ án
Tìm kiếm và lọc	Theo tên, danh mục, giá; sắp xếp theo giá hoặc mới nhất	Tìm theo tên, lọc theo danh mục và 4 khoảng giá, sắp theo giá hoặc mới nhất
Giỏ hàng và đặt hàng (BM_CART_CHECKOUT_01)	Tổng = tổng (số lượng x giá) - giảm giá + phí vận chuyển	Tính trong CheckoutService, đặt hàng trong một giao dịch; phí vận chuyển miễn phí
Voucher (BM_VOUCHER_01)	Giảm theo % hoặc số tiền; cho mọi sản phẩm hoặc một số sản phẩm; đơn tối thiểu; giới hạn lượt và thời gian	Bảng Vouchers và VoucherProducts; kiểm tra đủ các điều kiện khi tính tiền và khi đặt
Quản lý đơn (BM_ORDER_HIST	Khách xem lịch sử đơn; nhân viên cập nhật trạng thái	Trang Đơn của tôi; trang quản lý đơn theo một bảng quy tắc chuyển

Chức năng	Quy tắc	Hiện thực trong đồ án
ORY_01)		trạng thái
Giá và khuyến mãi (BM_PRICE_01)	Cập nhật hàng loạt, lưu lịch sử đổi giá, giá mới tự áp dụng	Trang Giá đổi hàng loạt theo % hoặc số tiền; bảng PriceChanges; khuyến mãi tự bật tắt theo thời gian
Báo cáo doanh thu (BM_REPORT_REVENUE_01)	Theo ngày, có sản phẩm bán chạy	Doanh thu theo ngày và theo sản phẩm; chỉ tính đơn đã thanh toán và không bị hủy; xuất Excel và PDF
Nhập kho (BM_STOCK_01)	Hàng nhập từ nhà cung cấp; kiểm tra trùng SKU; tồn cuối = tồn đầu + nhập	Phiếu nhập StockReceipts, mỗi dòng ghi tồn trước và sau; một biến thể xuất hiện hai lần trong phiếu thì bị từ chối
Bán hàng, hóa đơn (BM_INVOICE_01)	Tổng = tổng (số lượng x đơn giá) - giảm giá + phí vận chuyển + thuế; in hóa đơn hàng loạt	Đơn bán tại quầy là một đơn hàng loại Tại quầy; giá đã gồm VAT nên không có dòng thuế, không có phí vận chuyển; in nhiều hóa đơn cùng lúc từ danh sách đơn
Tra cứu đơn (BM_ORDER_TRACK_01)	Theo mã đơn và số điện thoại; trả về trạng thái và mã vận đơn; báo qua email	Trang Tra cứu đơn; email khi đặt hàng và khi giao cho đơn vị vận chuyển

3.1.3. Yêu cầu phi chức năng

Bảng 3.3: Yêu cầu phi chức năng

Nhóm	Yêu cầu trong đặc tả	Cách đáp ứng
Bảo mật	HTTPS; phiên đăng nhập an toàn; giới hạn số lần thử OTP và đăng nhập	Chuyển hướng HTTPS và HSTS; cookie của Identity (thay cho JWT vì đây là ứng dụng MVC dựng trang ở máy chủ); khóa tài khoản sau 5 lần sai; mã OTP đăng ký hủy sau 5 lần sai, mã đặt lại mật khẩu sai 5 lần thì khóa tài khoản; anti-forgery cho mọi form
Dễ sử dụng	Tìm kiếm, lọc và đặt hàng dễ dàng	Giao diện theo trang tham khảo rauvang.com; mọi lỗi có thông báo rõ và đường quay lại; dùng được trên điện thoại
Mở rộng	Thêm cổng thanh toán hoặc tính năng mới không phải làm lại	Cổng thanh toán qua giao diện IPaymentGateway; dịch vụ nghiệp vụ tách khỏi controller
Trao đổi dữ	Xuất Excel, PDF	Báo cáo xuất Excel bằng ClosedXML; báo

Nhóm	Yêu cầu trong đặc tả	Cách đáp ứng
liệu		cáo và hóa đơn in hoặc lưu PDF từ trình duyệt
Bảo trì	Ghi log rõ ràng	ILogger ở mọi chỗ xử lý lỗi cơ sở dữ liệu và email; tài liệu trong thư mục docs/

3.2. Đặc tả các trường hợp sử dụng

3.2.1. Danh sách use case và trạng thái

Đặc tả có 37 use case. Bảng 3.4 liệt kê toàn bộ cùng tình trạng hiện thực: cả 37 use case đều đã làm.

Bảng 3.4: Danh sách 37 use case

STT	Use case	Tác nhân	Trạng thái
1	Đăng ký tài khoản (OTP)	Khách	Đã làm
2	Gửi OTP qua email	Khách	Đã làm
3	Đăng nhập	Khách	Đã làm
4	Quên mật khẩu (OTP)	Khách	Đã làm
5	Đổi mật khẩu	Khách hàng	Đã làm
6	Cập nhật thông tin cá nhân	Khách hàng	Đã làm
7	Tìm kiếm sản phẩm	Khách	Đã làm
8	Lọc sản phẩm	Khách	Đã làm
9	Xem chi tiết sản phẩm	Khách	Đã làm
10	So sánh sản phẩm	Khách	Đã làm
11	Thêm sản phẩm vào giỏ	Khách hàng	Đã làm
12	Xóa sản phẩm khỏi giỏ	Khách hàng	Đã làm
13	Cập nhật số lượng trong giỏ	Khách hàng	Đã làm
14	Xem giỏ hàng	Khách hàng	Đã làm
15	Đặt hàng	Khách hàng	Đã làm
16	Áp dụng voucher	Khách hàng	Đã làm
17	Thanh toán (COD, VNPay, MoMo)	Khách hàng	Đã làm (VNPay, MoMo giả lập)
18	Tra cứu đơn hàng	Khách hàng	Đã làm

STT	Use case	Tác nhân	Trạng thái
19	Theo dõi vận chuyển	Khách hàng	Đã làm
20	Nhập hàng vào kho	Nhân viên, quản trị viên	Đã làm
21	Bán hàng tại quầy	Nhân viên, quản trị viên	Đã làm
22	Xác nhận đơn hàng	Nhân viên	Đã làm
23	Gán mã vận đơn	Nhân viên	Đã làm
24	Cập nhật trạng thái đơn	Nhân viên	Đã làm
25	Thêm sản phẩm mới	Quản trị viên	Đã làm
26	Sửa sản phẩm	Quản trị viên	Đã làm
27	Xóa sản phẩm (ngừng bán)	Quản trị viên	Đã làm
28	Tạo chương trình khuyến mãi	Quản trị viên, nhân viên	Đã làm
29	Thêm voucher	Quản trị viên	Đã làm
30	Sửa voucher	Quản trị viên	Đã làm
31	Xóa voucher	Quản trị viên	Đã làm
32	Gửi đánh giá sản phẩm	Khách hàng	Đã làm
33	Báo cáo tình hình kinh doanh	Nhân viên, quản trị viên	Đã làm
34	Báo cáo doanh thu	Nhân viên, quản trị viên	Đã làm
35	Xuất báo cáo (Excel, PDF)	Nhân viên, quản trị viên	Đã làm
36	In hóa đơn	Khách hàng, nhân viên	Đã làm (có in nhiều hóa đơn cùng lúc)
37	Gửi email xác nhận đơn	Hệ thống	Đã làm

Hình 3.1 gom 37 use case theo bốn tác nhân, cùng mức nhóm với Bảng 3.4 ở trên.



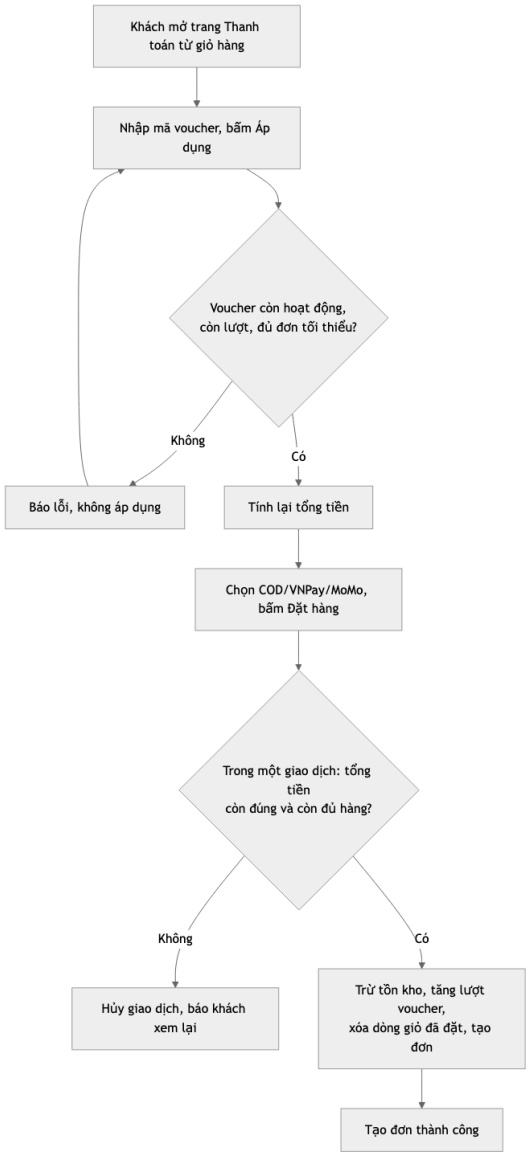
Hình 3.1: Sơ đồ use case theo tác nhân

3.2.2. Đặc tả use case Đăng ký tài khoản (use case 1, 2)

1. Khách mở trang Đăng ký, nhập email, họ tên, số điện thoại, mật khẩu và nhập lại mật khẩu.
2. Hệ thống kiểm tra dữ liệu hợp lệ, email và số điện thoại chưa được dùng, mật khẩu đủ 8 ký tự.
3. Hệ thống tạo mã OTP 6 chữ số có hạn 5 phút, giữ thông tin đăng ký tạm trong bộ nhớ đệm và gửi mã qua email. Tài khoản chưa được tạo ở bước này.
4. Khách nhập mã OTP. Nếu đúng, hệ thống tạo tài khoản qua UserManager của Identity và đăng nhập luôn.
5. Nếu nhập sai 5 lần, lần đăng ký bị hủy và khách được mời đăng ký lại.

3.2.3. Đặc tả use case Đặt hàng và áp dụng voucher (use case 15, 16)

1. Khách hàng đã đăng nhập mở trang Thanh toán từ giỏ hàng; địa chỉ mặc định được điền sẵn.
2. Khách nhập mã voucher và bấm Áp dụng; hệ thống kiểm tra voucher còn hoạt động, còn trong thời gian, còn lượt, đủ giá trị đơn tối thiểu và áp cho đúng sản phẩm, rồi tính lại tổng.
3. Khách chọn phương thức thanh toán (COD, VNPay, MoMo) và bấm Đặt hàng.
4. Trong một giao dịch, hệ thống tính lại giá, so với tổng khách đã thấy, trừ tồn kho có điều kiện, tăng lượt dùng voucher có điều kiện, xóa các dòng giỏ đã đặt và tạo đơn.
5. Nếu tổng tiền đã thay đổi (ví dụ khuyến mãi vừa bắt đầu hoặc kết thúc) hoặc hết hàng, giao dịch bị hủy và khách được báo để xem lại; bấm đặt hai lần chỉ tạo một đơn.



Hình 3.2: Lưu đồ đặt hàng và áp dụng voucher

3.2.4. Đặc tả use case Thanh toán trực tuyến (use case 17)

1. Khách chọn VNPay hoặc MoMo; hệ thống tạo một lần thanh toán và chuyển khách sang trang cổng thanh toán (giả lập).
2. Cổng trả kết quả về kèm chữ ký HMAC-SHA512; hệ thống kiểm tra chữ ký và số tiền.
3. Chỉ khi chữ ký đúng, số tiền khớp và lần thanh toán còn đang chờ thì đơn mới được đánh dấu đã thanh toán; kết quả gửi lại lần hai không được tính hai lần.
4. Nếu thanh toán thất bại hoặc bị hủy, trang đơn hàng hiện nút Thanh toán lại; đơn đã hủy thì không thể thanh toán.

3.2.5. Đặc tả use case Xử lý đơn hàng (use case 22, 23, 24)

Bảng 3.5: Quy tắc chuyển trạng thái đơn hàng (OrderTransitions)

Hành động	Nhân viên làm được khi đơn ở trạng thái	Khách hàng làm được khi	Trạng thái mới
Xác nhận	Chờ xác nhận (đơn trực tuyến phải đã thanh toán)	Không	Đã xác nhận
Giao vận chuyển (kèm đơn vị và mã vận đơn)	Đã xác nhận	Không	Đang giao
Hoàn tất	Đang giao (đơn COD được ghi nhận đã thanh toán)	Không	Hoàn tất
Hủy	Chờ xác nhận hoặc Đã xác nhận	Chờ xác nhận	Đã hủy (trả lại tồn kho và lượt voucher)

Hai nhân viên bấm cùng một nút, hoặc khách hủy đúng lúc nhân viên xác nhận, chỉ có một thay đổi được áp dụng vì mỗi lần chuyển trạng thái là một cập nhật có điều kiện trên trạng thái cũ. Đơn bán tại quầy được tạo ở trạng thái Hoàn tất nên không ai hủy hay thanh toán lại được.

Khách tra cứu đơn không cần đăng nhập, bằng mã đơn và số điện thoại người nhận. Nhập sai số điện thoại 5 lần liên tiếp cho cùng một mã đơn thì mã đơn đó bị khóa tra cứu 5 phút, cùng con số mà khóa tài khoản của Identity dùng; nhập đúng thì số lần sai về 0.



Hình 3.3: Lưu đồ trạng thái đơn hàng (OrderTransitions)

3.2.6. Đặc tả use case Nhập hàng vào kho (use case 20)

1. Nhân viên mở trang Nhập hàng; hệ thống chuyển sang địa chỉ có mã form riêng (/Admin/Stock/New?key=...).
2. Nhân viên nhập tên nhà cung cấp, chọn từng sản phẩm, số lượng và giá nhập; nút Thêm dòng thêm dòng mới ngay trên trang.
3. Hệ thống kiểm tra: có nhà cung cấp, có ít nhất một dòng, số lượng từ 1 trở lên, giá nhập từ 0 tới giá lớn nhất được lưu, không có biến thể nào xuất hiện hai lần (kiểm tra trùng SKU của BM_STOCK_01).
4. Trong một giao dịch, mỗi dòng cộng số lượng vào tồn kho ngay trong cơ sở dữ liệu (StockQty + n) rồi đọc lại tồn sau; tồn trước được tính bằng tồn sau trừ số lượng, nên một đơn bán chen vào đúng lúc đó vẫn được giữ và công thức tồn cuối = tồn đầu + nhập luôn đúng.
5. Phiếu nhập được lưu kèm người lập và thời điểm; nhân viên được chuyển tới trang phiếu, có thể in phiếu.
6. Bấm lưu lần hai, bấm Quay lại của trình duyệt rồi lưu, hay mở trùng tab đều không nhập hàng hai lần, vì mã form là duy nhất và nằm trong địa chỉ trang.

3.2.7. Đặc tả use case Bán hàng tại quầy (use case 21)

1. Nhân viên mở trang Bán hàng tại quầy (cũng có mã form trong địa chỉ), chọn sản phẩm và số lượng, có thể nhập mã voucher và thông tin khách.
2. Nhân viên bấm Tính tiền: hệ thống tính giá đang bán (có khuyến mãi), áp voucher theo đúng quy tắc của bán trực tuyến và báo các vấn đề như hết hàng hoặc voucher không dùng được.
3. Nút Hoàn tất bán chỉ hiện khi bảng tính tiền không có vấn đề nào. Nhân viên chọn tiền mặt hoặc chuyển khoản rồi bấm Hoàn tất bán.
4. Hệ thống chỉ bán đúng những dòng đã tính tiền và đúng tổng đã hiển thị; nếu giá, khuyến mãi hay các dòng đã đổi thì yêu cầu tính lại.
5. Trong một giao dịch, hệ thống trừ tồn kho có điều kiện, tăng lượt dùng voucher có điều kiện, tạo một đơn hàng loại Tại quầy ở trạng thái Hoàn tất, Đã thanh toán, kèm lần thanh toán và người bán.
6. Khách lẻ không cần tài khoản; nếu nhập email của một tài khoản có sẵn, đơn được gắn vào tài khoản đó và hiện trong Đơn của tôi. Giá đã gồm VAT nên hóa đơn ghi "Giá đã bao gồm VAT" và không có dòng thuế.

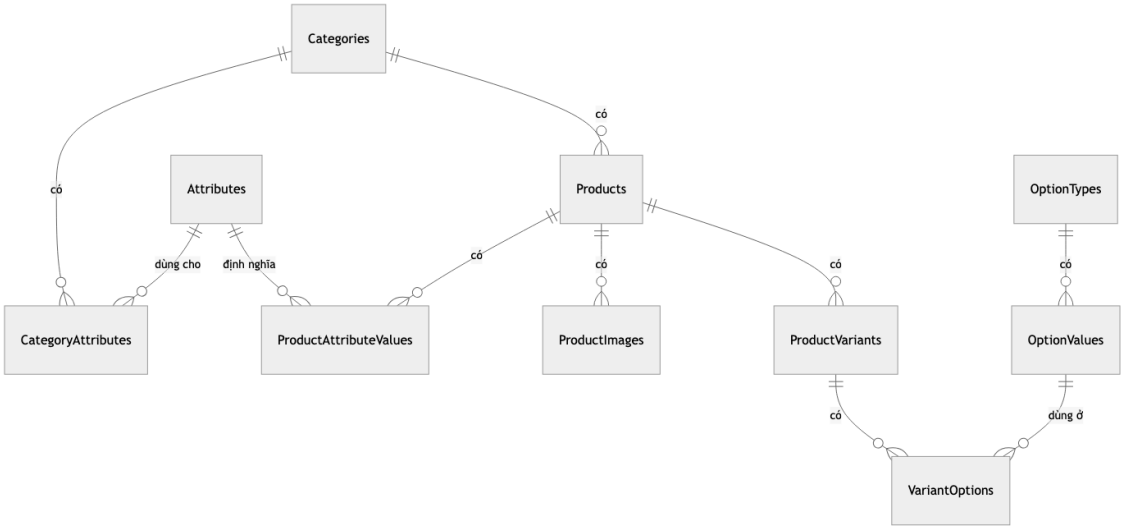
3.3. Thiết kế cơ sở dữ liệu

3.3.1. Tổng quan

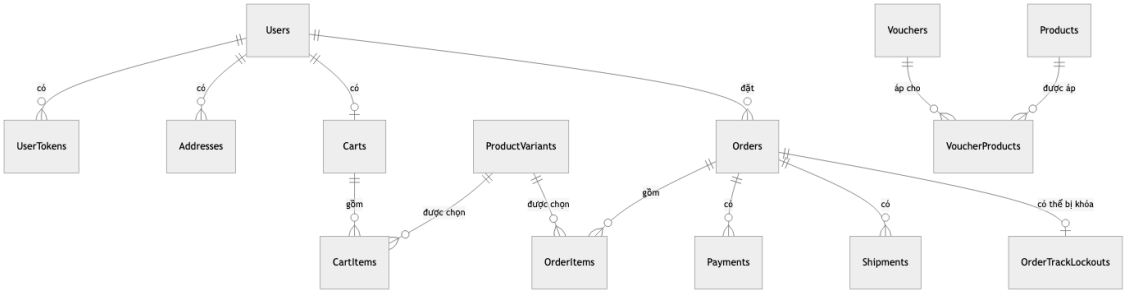
Lược đồ gồm 24 bảng theo thiết kế của đặc tả, tên cột giữ nguyên, cộng thêm sáu bảng do nghiệp vụ cần: Vouchers, VoucherProducts, PriceChanges, StockReceipts, StockReceiptLines và OrderTrackLockouts. Các nhóm bảng chính:

- Tài khoản: Users, UserTokens, Addresses.
- Danh mục và sản phẩm: Categories, Products, ProductVariants, OptionTypes, OptionValues, VariantOptions, ProductImages, Attributes, CategoryAttributes, ProductAttributeValues.
- Mua hàng: Carts, CartItems, Orders, OrderItems, Payments, Shipments, Vouchers, VoucherProducts.
- Tương tác: Reviews, ReviewMedia, Favorites, CompareLists, CompareItems.
- Giá và kho: PriceChanges, StockReceipts, StockReceiptLines.
- Bảo vệ tra cứu đơn: OrderTrackLockouts.

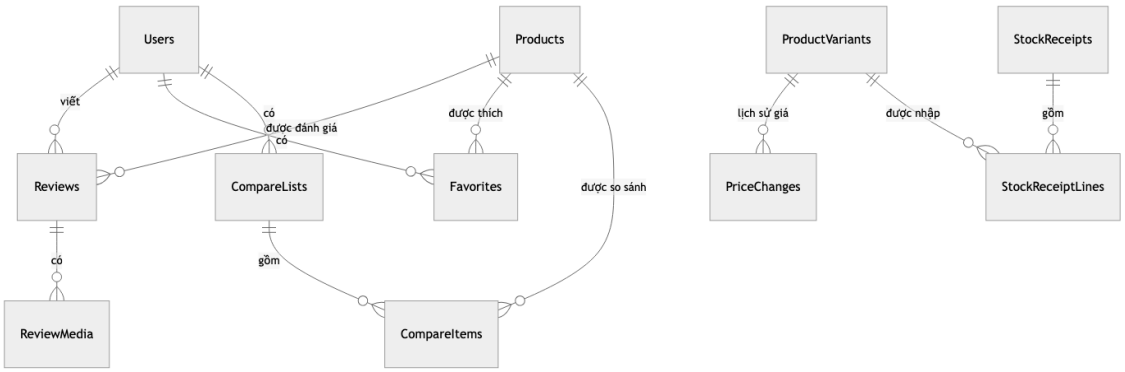
Hình 3.4 đến 3.6 vẽ lại ba nhóm bảng trên thành sơ đồ thực thể - quan hệ (ERD), tách thành ba hình cho dễ đọc thay vì gộp 24 bảng vào một sơ đồ duy nhất.



Hình 3.4: ERD: Danh mục và sản phẩm



Hình 3.5: ERD: Tài khoản và mua hàng



Hình 3.6: ERD: Tương tác, giá và kho

3.3.2. Các bảng chính

Bảng 3.6: Bảng Products và ProductVariants

Bảng	**Các cột chính**	**Ghi chú**
Products	Id, CategoryId, Name, Slug,	Một mẫu máy; Status = đang

Bảng	Các cột chính	Ghi chú
	Description, BasePrice, Status, SortOrder, CreatedAt, UpdatedAt	bán; ngừng bán thay cho xóa để đơn cũ còn nguyên
ProductVariants	Id, ProductId, SKU, Price, StockQty, Status, CreatedAt, UpdatedAt	Một biến thể (cấu hình, màu, khu vực); Price rỗng nghĩa là Liên hệ; UpdatedAt dùng làm phiên bản để chống ghi đè
VariantOptions, OptionTypes, OptionValues	VariantId, OptionTypeId, OptionValueId	Gắn cấu hình (config), màu (color), khu vực (region) cho từng biến thể

Bảng 3.7: Bảng đơn hàng và thanh toán

Bảng	Các cột chính	Ghi chú
Orders	Id, UserId, Channel, SoldByUserId, FormKey, OrderStatus, PaymentStatus, Subtotal, DiscountAmount, ShippingFee, TotalAmount, VoucherCode, thông tin người nhận, Note, CreatedAt, UpdatedAt	Trạng thái: Chờ xác nhận, Đã xác nhận, Đang giao, Hoàn tất, Đã hủy. Channel: Trực tuyến hoặc Tại quầy
OrderItems	Id, OrderId, ProductId, VariantId, Price, Quantity	Price là giá thực tính lúc đặt (đã trừ khuyến mãi)
Payments	Id, OrderId, Method, Status, TxnId, PaidAmount, PaidAt, ProviderRaw, CreatedAt	Mỗi lần thanh toán một dòng; giữ dữ liệu gốc cổng trả về. Method: COD, VNPay, MoMo, tiền mặt, chuyển khoản
Shipments	Id, OrderId, Carrier, TrackingNo, Status, Fee	Đơn vị vận chuyển và mã vận đơn

Bảng 3.8: Các bảng thêm so với đặc tả

Bảng hoặc cột	Nội dung	Lý do
Vouchers, VoucherProducts	Mã, loại giảm (% hoặc số tiền), mức giảm, đơn tối thiểu, thời gian, giới hạn lượt, số lượt đã dùng, sản phẩm áp dụng	Đặc tả chỉ lưu mã voucher dạng chữ trong Orders nhưng quy tắc BM_VOUCHER_01 cần đủ các thông tin này
Vouchers.Kind,	Kind = Code (voucher nhập mã)	Khuyến mãi dùng chung

Bảng hoặc cột	Nội dung	Lý do
Vouchers.Name	hoặc Automatic (khuyến mãi tự áp dụng); Name là tên chương trình	bảng với voucher để không phải chép lại trang quản lý
PriceChanges	Biến thể, giá cũ, giá mới, nguồn thay đổi, người đổi, thời điểm	BM_PRICE_01 yêu cầu lưu lịch sử đổi giá
StockReceipts, StockReceiptLines	Phiếu: nhà cung cấp, ghi chú, mã form (duy nhất), người lập, thời điểm. Dòng: biến thể, số lượng, giá nhập, tồn trước, tồn sau	Use case 20 và BM_STOCK_01 cần phiếu nhập; đặc tả chưa có bảng nào để lưu
Orders: UserId cho phép rỗng; Channel, SoldByUserId, FormKey	Khách lẻ không có tài khoản; kênh bán; nhân viên bán; mã form chống bán trùng	Use case 21: lần bán tại quầy được lưu như một đơn hàng để dùng chung hóa đơn, báo cáo doanh thu và quy tắc trừ kho
PaymentMethod: Cash, BankTransfer	Tiền mặt, chuyển khoản tại quầy	Hai cách trả tiền khi mua tại cửa hàng
Users: NormalizedEmail, SecurityStamp, AccessFailedCount, LockoutEnd	Các cột Identity cần	Dùng ASP.NET Core Identity trên bảng Users của đặc tả
Reviews: Reply, RepliedAt, UpdatedAt	Trả lời của cửa hàng	Đặc tả cho nhân viên trả lời đánh giá nhưng chưa có cột lưu
OrderTrackLockouts	Mã đơn, số lần nhập sai số điện thoại liên tiếp, thời điểm hết khóa	Trang tra cứu đơn công khai cần chặn đoán số điện thoại; số lần sai phải được lưu lại giữa các yêu cầu

3.3.3. Mối quan hệ giữa các bảng

- Một danh mục có nhiều sản phẩm; một sản phẩm có nhiều biến thể và nhiều ảnh.
- Một người dùng có một giỏ hàng (chỉ mục duy nhất trên UserId), nhiều địa chỉ, nhiều đơn hàng và nhiều đánh giá.
- Một giỏ hàng có nhiều dòng, mỗi biến thể chỉ một dòng (chỉ mục duy nhất trên CartId và VariantId).

- Một đơn hàng có nhiều dòng hàng, nhiều lần thanh toán và một hoặc nhiều lần vận chuyển. Đơn bán tại quầy có thể không gắn với tài khoản nào (khách lẻ) và gắn với nhân viên bán.
- Một phiếu nhập có nhiều dòng, mỗi biến thể chỉ một dòng (chỉ mục duy nhất trên ReceiptId và VariantId); biến thể đã có trong phiếu nhập thì không xóa được.
- Mỗi khách hàng chỉ có một đánh giá cho một sản phẩm (chỉ mục duy nhất trên ProductId và UserId).
- Một voucher hoặc khuyến mãi có thể áp cho một số sản phẩm qua VoucherProducts; không có dòng nào nghĩa là áp cho mọi sản phẩm.
- Mỗi đơn hàng có nhiều nhất một dòng khóa tra cứu (OrderTrackLockouts); mã đơn không có thật thì không bao giờ có dòng khóa.

3.3.4. Một số điều chỉnh so với thiết kế gốc

- Giá sản phẩm và biến thể cho phép rỗng, nghĩa là "Liên hệ", thay vì lưu số 0 có thể bị cộng nhầm vào tổng đơn.
- Trạng thái đơn hàng có 5 giá trị (thêm Đã xác nhận) vì đặc tả nêu miền giá trị từ 0 đến 4 nhưng chỉ đặt tên 4 giá trị.
- Mã OTP đăng ký giữ trong bộ nhớ đệm chứ không lưu vào UserTokens, vì tài khoản chỉ được tạo sau khi OTP đúng. Mã đặt lại mật khẩu được lưu trong UserTokens dưới dạng đã băm.
- Danh sách so sánh lưu trong cookie để khách chưa đăng nhập cũng dùng được, nên hai bảng CompareLists và CompareItems được giữ nguyên nhưng chưa dùng.
- Năm sản phẩm chưa có ảnh được phép sử dụng của đúng mẫu máy hiển thị một hình vẽ theo loại sản phẩm (đồng hồ, máy tính bảng, pin sạc...) thay cho ảnh của đời máy trước.

3.4. Thiết kế kiến trúc

Bảng 3.9: Các dự án trong giải pháp

Dự án	**Nội dung**	**Phụ thuộc**
AppleStore.Domain	Các lớp entity và enum, không phụ thuộc thư viện nào	Không
AppleStore.Infrastructure	AppDbContext, cấu hình EF Core, migration, Identity (UserStore), các dịch vụ nghiệp vụ, thanh toán, email	Domain

Dự án	Nội dung	Phụ thuộc
AppleStore.Web	Controller, view Razor, Area Admin, view component, wwwroot, Program.cs	Infrastructure, Domain
AppleStore.Tests	Test xUnit cho dịch vụ và cho toàn ứng dụng web	Web, Infrastructure, Domain

Một yêu cầu đi qua các bước: trình duyệt gửi yêu cầu, ASP.NET Core định tuyến tới controller, model binding tạo đối tượng từ form, controller gọi dịch vụ nghiệp vụ trong Infrastructure, dịch vụ đọc ghi qua AppDbContext, controller trả về view Razor hoặc chuyển hướng kèm thông báo.

Một số quy tắc được gom về một nơi duy nhất để mọi trang không thể hiểu khác nhau: OrderTransitions cho chuyển trạng thái đơn, DiscountMath cho công thức giảm giá, SalePrices cho giá đang bán, SaleRules cho kiểm tra voucher, trừ kho và tăng lượt voucher (dùng chung cho đặt hàng trực tuyến và bán tại quầy), PriceLimits cho giá lớn nhất được lưu, FormKeys cho mã form chống lưu trùng (dùng chung cho phiếu nhập và bán tại quầy), VariantText cho tên biến thể trên các trang nhân viên, NoPhoto cho hình thay thế khi chưa có ảnh.

3.5. Thiết kế các quy tắc tính tiền

- Giá đang bán của một biến thể = giá gốc trừ mức giảm lớn nhất trong các khuyến mãi đang chạy áp cho sản phẩm đó; một mức giảm làm giá về 0 thì không được áp dụng.
- Tạm tính = tổng (số lượng x giá đang bán) của các dòng mua được.
- Giảm giá voucher tính trên tạm tính đã trừ khuyến mãi; giảm theo % được làm tròn tới đồng, không vượt quá phần được áp dụng.
- Tổng tiền = tạm tính - giảm giá voucher + phí vận chuyển (hiện miễn phí). Giá bán đã bao gồm VAT nên không cộng thêm thuế; đơn bán tại quầy không có phí vận chuyển.
- Doanh thu trong báo cáo chỉ tính các đơn đã thanh toán và không bị hủy, gom theo ngày giờ Việt Nam (UTC+7).
- Đổi giá hàng loạt chỉ áp cho biến thể đang bán; theo % thì giá mới được làm tròn tới 1.000 đồng; nếu có biến thể nào về 0 hoặc vượt giá lớn nhất thì không thay đổi gì. Mỗi lần bấm Đổi giá là một lần đổi riêng và đều được ghi vào lịch sử giá.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM

4.1. Môi trường phát triển

Bảng 4.1: Phần mềm và công cụ

Thành phần	Phiên bản hoặc ghi chú
Hệ điều hành	macOS
Nền tảng	.NET 10 (SDK 10.0.400)
Web framework	ASP.NET Core MVC, Razor
ORM	Entity Framework Core 10.0.12, Code First
Cơ sở dữ liệu	SQLite (khi phát triển); lược đồ theo kiểu SQL Server
Kiểm thử	xUnit 2.9.3, Microsoft.AspNetCore.Mvc.Testing 10.0.12, playwright-cli với Chromium
Quản lý mã nguồn	Git, GitHub (pull request vào nhánh dev)

4.2. Cấu trúc mã nguồn

Bảng 4.2: Các thành phần chính trong mã nguồn

Thư mục	Nội dung
src/AppleStore.Web/Controllers	Account, Addresses, Products, Cart, Checkout, Payments, PaymentSimulator, Orders, Track, Compare, Reviews, Home
src/AppleStore.Web/Areas/ Admin/Controllers	Dashboard, Orders, Products, Vouchers, Promotions (chung lớp nền DiscountAdminController), Prices, Stock, Sales, Reports, Reviews, Users
src/AppleStore.Infrastructure/ Services	CartService, CheckoutService, OrderManagementService, ProductCatalogService, CompareService, ReviewService, ReportService, PriceService, SalePrices, SaleRules, StockService, InStoreSaleService, AdminCatalogService, AdminVoucherService, AdminUserService, RegistrationService, PasswordResetService
src/AppleStore.Infrastructure/ Payments	PaymentService, SimulatedPaymentGateway, PaymentSignature (HMAC-SHA512)
src/AppleStore.Infrastructure/ Identity	UserStore, AdminSeeder, AppUserClaimsPrincipalFactory
tests/AppleStore.Tests	647 test xUnit

Thư mục	Nội dung
setup/verify-*	16 kịch bản kiểm tra trên trình duyệt; setup/screenshots chụp các hình trong chương này; setup/report dựng file báo cáo này

Cách chạy ứng dụng: dotnet tool restore, dotnet ef database update (tạo cơ sở dữ liệu và dữ liệu mẫu), rồi dotnet run --project src/AppleStore.Web. Hướng dẫn đầy đủ có trong README.md.

4.2.1. Một số module chính (kèm mã minh họa)

Bốn đoạn mã dưới đây là phần lõi, rút gọn, của bốn cơ chế đã nêu ở Chương 2 và Chương 3; mỗi đoạn lấy nguyên văn từ mã nguồn thật, bỏ bớt phần còn lại của file để vừa trang in.

WholeDongAttribute (src/AppleStore.Web/Models/Admin/AdminForms.cs) là thuộc tính kiểm tra dữ liệu dùng chung cho mọi ô số tiền nhân viên gõ tay: giá sản phẩm, giá biến thể, mức giảm và đơn tối thiểu của voucher, giá nhập kho. Tham số gateProperty/gateValue (ví dụ chỉ bắt buộc khi Type == Fixed) để một thuộc tính tùy theo giá trị của thuộc tính khác.

AdminForms.cs — AppleStore.Infrastructure

```
[AttributeUsage(AttributeTargets.Property)]
public sealed class WholeDongAttribute : ValidationAttribute
{
    private readonly string? _gateProperty;
    private readonly object? _gateValue;

    public WholeDongAttribute() { }

    public WholeDongAttribute(string gateProperty, object gateValue)
    {
        _gateProperty = gateProperty;
        _gateValue = gateValue;
    }

    protected override ValidationResult? IsValid(
        object? value, ValidationContext validationContext)
    {
        if (_gateProperty is not null)
        {
            var gate = validationContext.ObjectType
                .GetProperty(_gateProperty)?
                .GetValue(validationContext.ObjectInstance);
            if (!Equals(gate, _gateValue))
                return ValidationResult.Success;
        }
        return value as decimal? is not { } v || v.Scale == 0
            ? ValidationResult.Success
            : new ValidationResult("Type numbers only, without dots or commas.");
    }
}
```

Hình 4.1: WholeDongAttribute, thuộc tính kiểm tra số tiền nguyên đồng

OrderTransitions (src/AppleStore.Infrastructure/Services/OrderTransitions.cs) là bảng quy tắc duy nhất cho việc chuyển trạng thái đơn hàng (Hình 3.6 vẽ lại đúng bảng này); mọi trang (xác nhận, giao, hoàn tất, hủy) đều gọi vào đây thay vì mỗi trang tự kiểm tra trạng thái theo cách riêng.

OrderTransitions.cs — AppleStore.Infrastructure

```
public static class OrderTransitions
{
    private static readonly Dictionary<OrderAction,
        (OrderStatus[] Staff, OrderStatus[] Customer, OrderStatus To)> Rules = new()
    {
        [OrderAction.Confirm] =
            ([OrderStatus.Pending], [], OrderStatus.Confirmed),
        [OrderAction.Ship] =
            ([OrderStatus.Confirmed], [], OrderStatus.Shipping),
        [OrderAction.Complete] =
            ([OrderStatus.Shipping], [], OrderStatus.Completed),
        [OrderAction.Cancel] = ([OrderStatus.Pending, OrderStatus.Confirmed],
            [OrderStatus.Pending], OrderStatus.Cancelled),
    };

    public static IReadOnlyList<OrderStatus> From(OrderAction action, bool byStaff) =>
        byStaff ? Rules[action].Staff : Rules[action].Customer;

    public static OrderStatus To(OrderAction action) => Rules[action].To;
}
```

Hình 4.2: OrderTransitions, bảng quy tắc chuyển trạng thái đơn hàng

PriceService (src/AppleStore.Infrastructure/Services/PriceService.cs) đổi giá hàng loạt trong một giao dịch EF Core: chỉ lấy biến thể đang bán (v.Status && v.Product.Status, sửa lỗi BM_PRICE_01 nêu ở Chương 4.5), rồi ghi từng giá bằng cập nhật có điều kiện (so Price và UpdatedAt cũ trong mệnh đề Where) để hai người đổi giá cùng lúc không ghi đè nhau.

PriceService.cs — AppleStore.Infrastructure

```
await using var tx = await _db.Database.BeginTransactionAsync(ct);

var variants = await _db.ProductVariants.AsNoTracking()
    .Where(v => v.Price != null && v.Status && v.Product.Status
        && (products.Contains(v.ProductId)
            || v.Product.CategoryId == input.CategoryId))
    .Select(v => new { v.Id, v.SKU, Price = v.Price!.Value, v.UpdatedAt })
    .ToListAsync(ct);

var changes = variants
    .Select(v => (v.Id, Old: v.Price, New: NewPrice(input, v.Price), v.UpdatedAt))
    .Where(c => c.New != c.Old)
    .ToList();

if (changes.Any(c => c.New <= 0))
    return new PriceBatchResult(PriceBatchOutcome.PriceTooLow);

foreach (var c in changes)
{
    var saved = await _db.ProductVariants
        .Where(v => v.Id == c.Id
            && v.Price == c.Old && v.UpdatedAt == c.UpdatedAt)
        .ExecuteUpdateAsync(s => s
            .SetProperty(v => v.Price, c.New)
            .SetProperty(v => v.UpdatedAt, now), ct);

    if (saved != 1) { await tx.RollbackAsync(ct); return NothingChanged; }
}

await tx.CommitAsync(ct);
```

Hình 4.3: PriceService, đổi giá hàng loạt trong một giao dịch (rút gọn)

UserStore (src/AppleStore.Infrastructure/Identity/UserStore.cs) là chỗ ASP.NET Core Identity đọc và ghi dữ liệu tài khoản; lớp này cài các interface của Identity để đọc/ghi thẳng vào bảng Users của đặc tả, không tạo bảng AspNetUsers riêng.

UserStore.cs — AppleStore.Infrastructure

```
public class UserStore :
    IUserPasswordStore<User>,
    IUserEmailStore<User>,
    IUserSecurityStampStore<User>,
    IUserLockoutStore<User>
{
    private readonly AppDbContext _db;

    public UserStore(AppDbContext db) => _db = db;

    public Task<string> GetUserIdAsync(User user, CancellationToken ct) =>
        Task.FromResult(user.Id.ToString());

    public async Task<User?> FindByIdAsync(string userId, CancellationToken ct) =>
        int.TryParse(userId, out var id)
            ? await _db.Users.FindAsync([id], ct)
            : null;
}
```

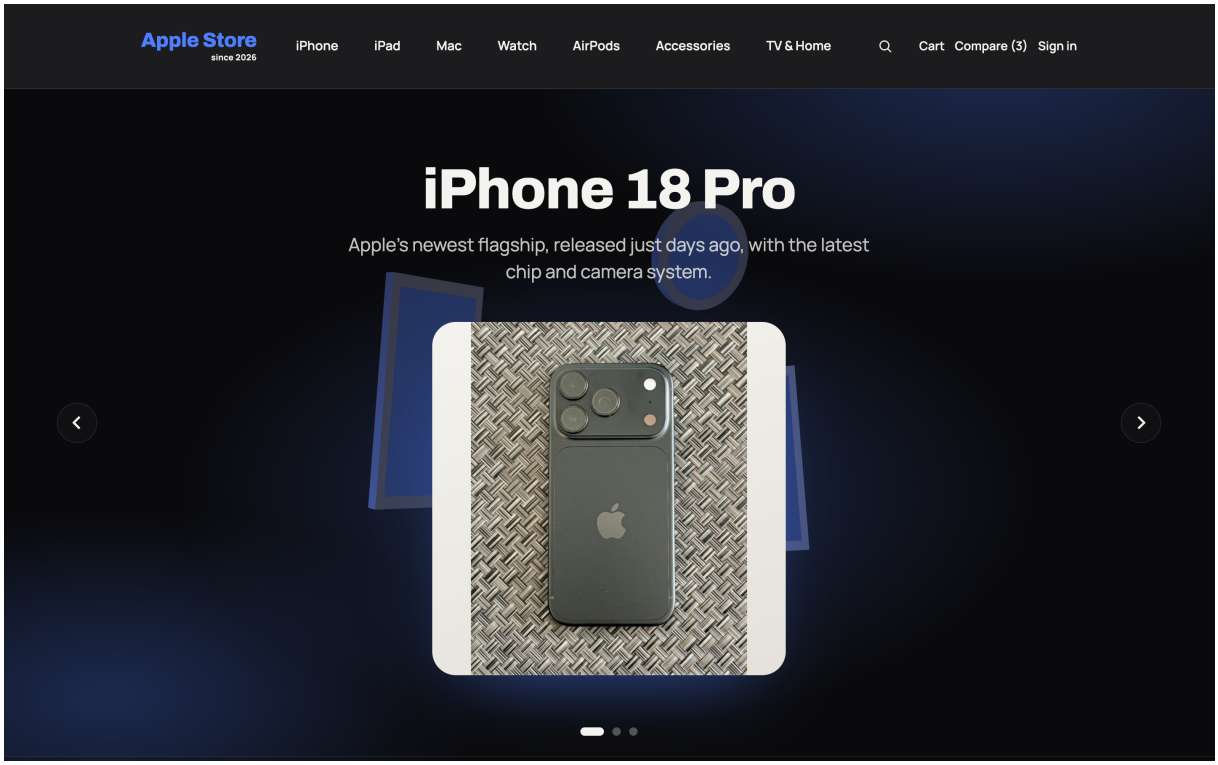
Hình 4.4: UserStore, Identity đọc ghi trên bảng Users của đặc tả

4.3. Giao diện phía khách hàng

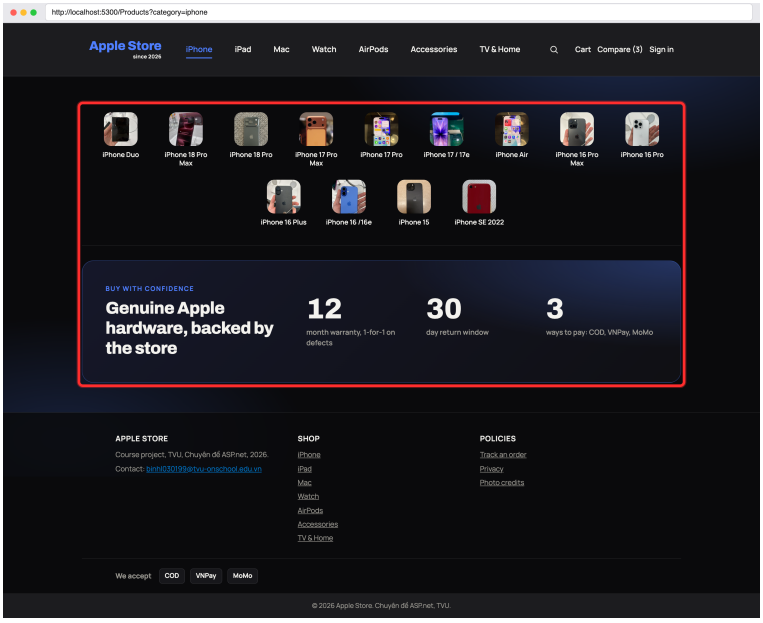
Các hình dưới đây được chụp từ ứng dụng thật, chạy trên một cơ sở dữ liệu mới tạo bằng migration, bằng script setup/screenshots/capture.py. Mỗi hình được cắt đúng khối cần xem và chụp ở độ phân giải gấp đôi để chữ đọc rõ khi in.

4.3.1. Trang chủ và danh mục sản phẩm

Trang chủ có băng chuyền động giới thiệu sản phẩm nổi bật và các banner sản phẩm. Trang danh mục hiển thị dải mẫu máy của danh mục, giống cách trình bày của trang tham khảo rauvang.com.

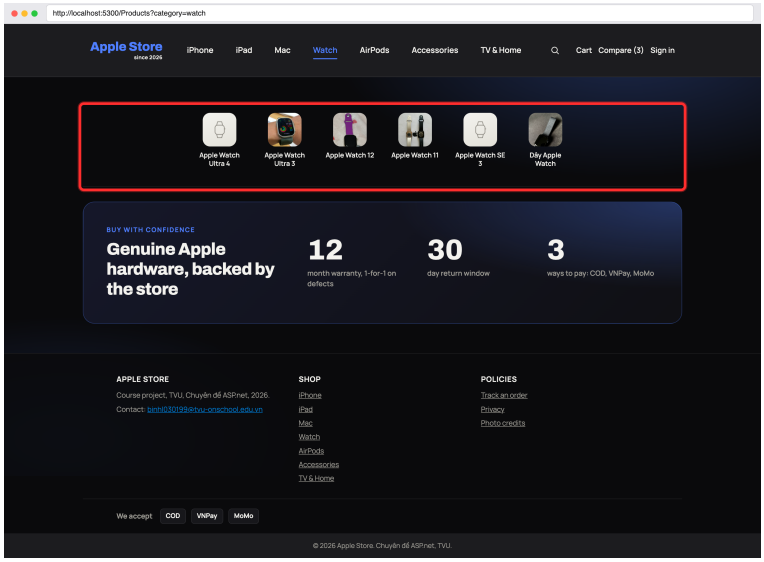


Hình 4.5: Trang chủ



Hình 4.6: Trang danh mục iPhone (khung đỏ đánh dấu khối chính)

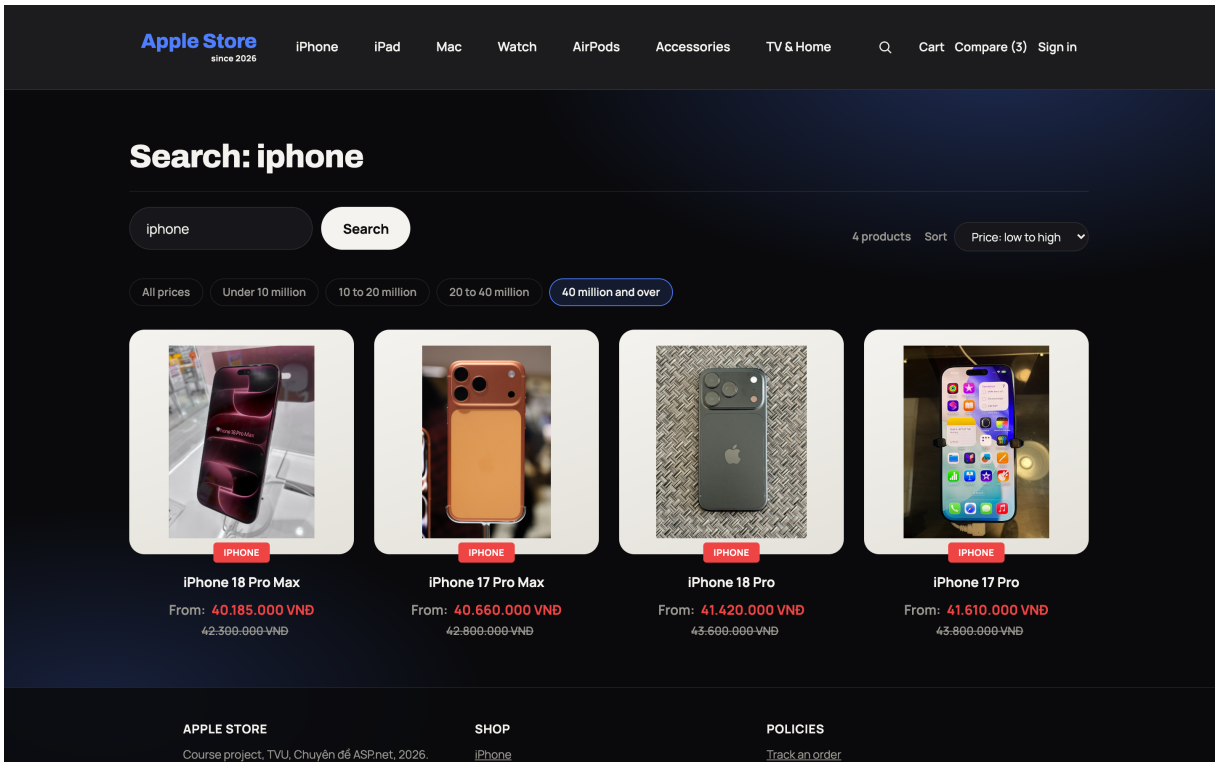
Năm sản phẩm chưa có ảnh được phép sử dụng của đúng mẫu máy hiển thị hình vẽ theo loại sản phẩm, ví dụ hình đồng hồ cho Apple Watch, thay vì ảnh của đời máy trước.



Hình 4.7: Dải mẫu Apple Watch: hai mẫu chưa có ảnh hiện hình vẽ đồng hồ (khung đỏ đánh dấu khối chính)

4.3.2. Tìm kiếm, lọc theo giá và sắp xếp

Khách tìm theo tên, lọc theo bốn khoảng giá (dưới 10 triệu, 10 đến 20 triệu, 20 đến 40 triệu, từ 40 triệu) và sắp theo giá hoặc mới nhất. Khoảng giá đang chọn được giữ lại khi đổi cách sắp xếp hoặc tìm kiếm.



Hình 4.8: Tìm kiếm kết hợp lọc theo khoảng giá và sắp xếp

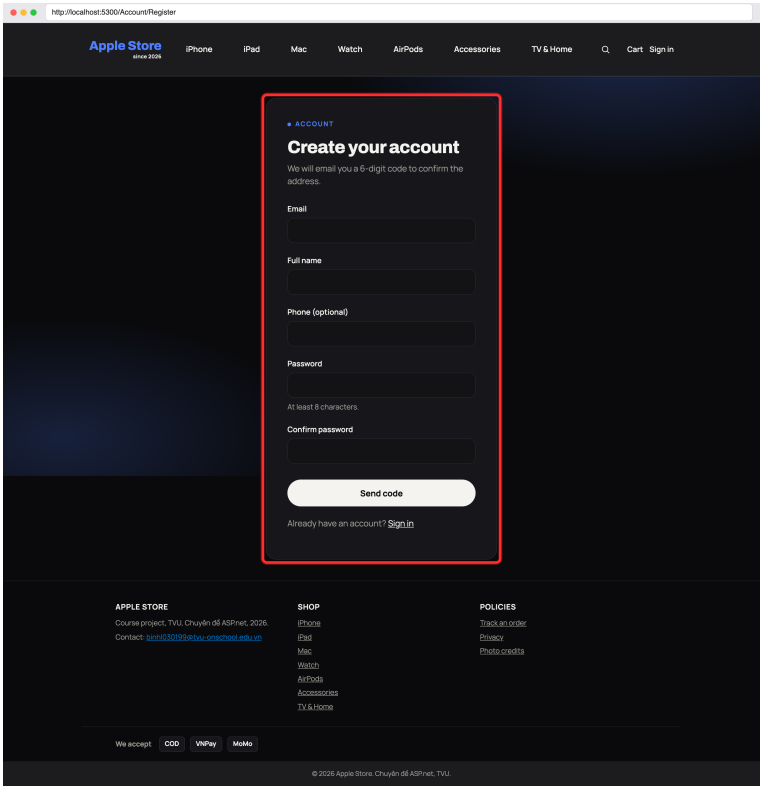
4.3.3. Trang cấu hình sản phẩm

Mỗi cấu hình có trang riêng với các nút chọn màu và khu vực; giá, mã SKU và tồn kho đổi ngay khi chọn. Khi có khuyến mãi, giá cũ được gạch ngang và tên chương trình hiện bên cạnh.

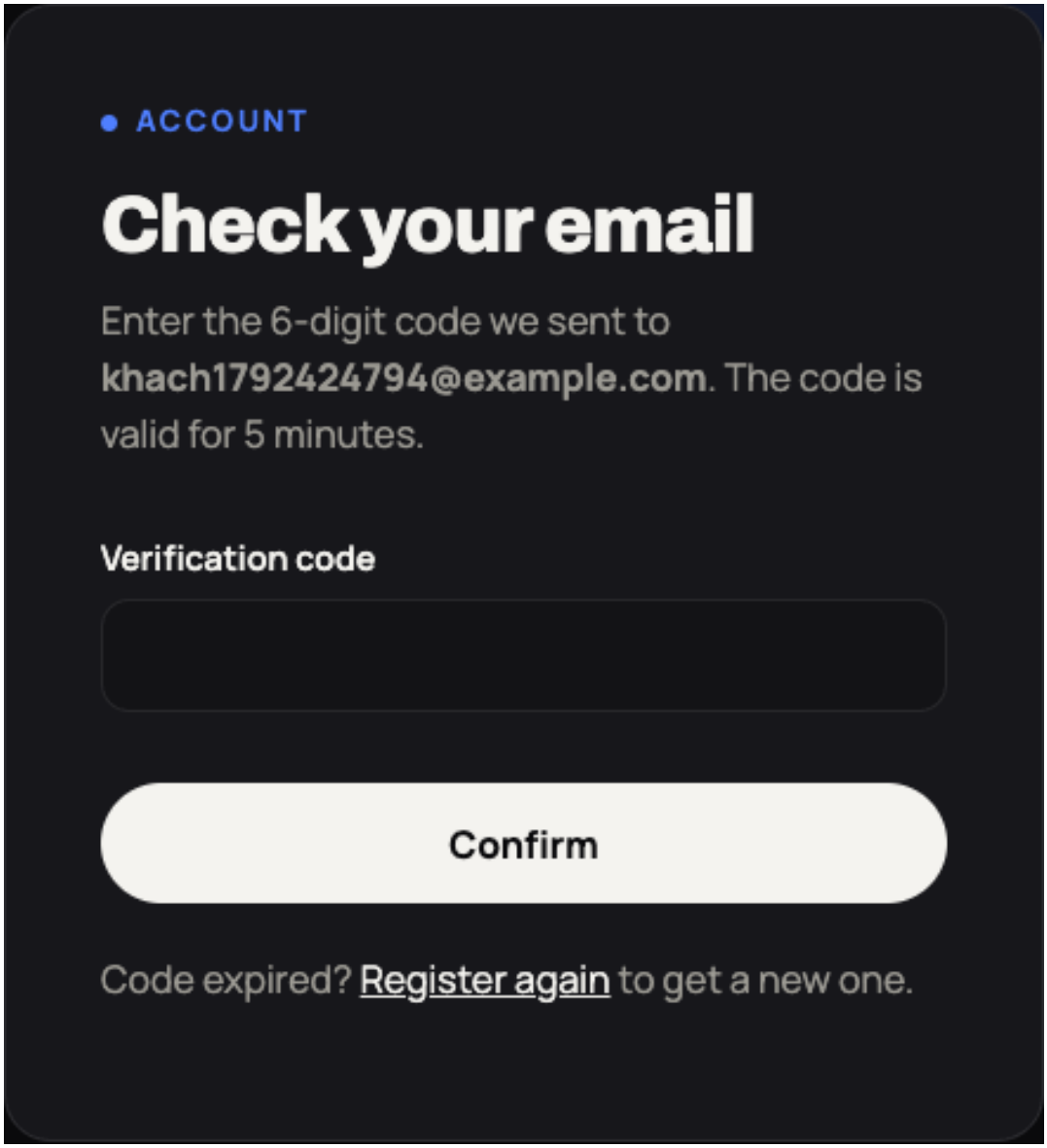


Hình 4.9: Trang cấu hình sản phẩm có khuyến mãi (khung đỏ đánh dấu khối chính)

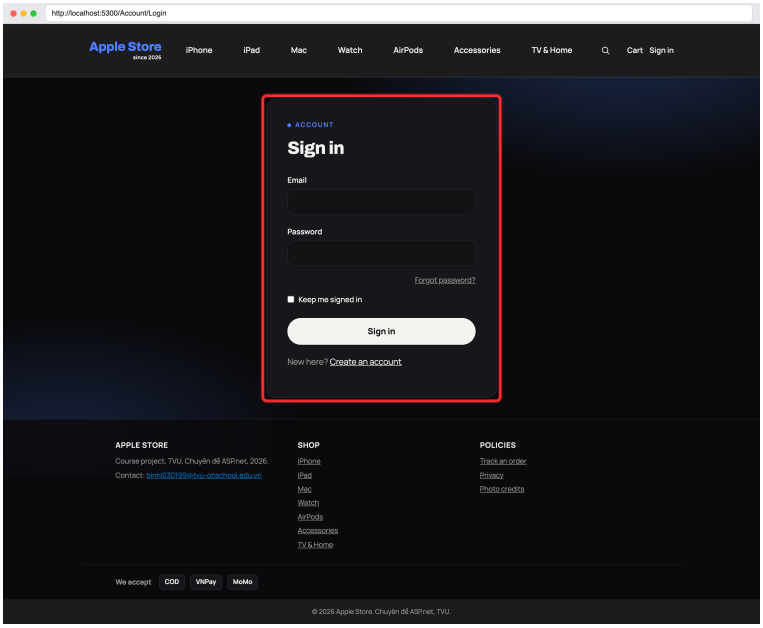
4.3.4. Đăng ký, xác thực OTP và đăng nhập



Hình 4.10: Trang đăng ký tài khoản (khung đỏ đánh dấu khối chính)



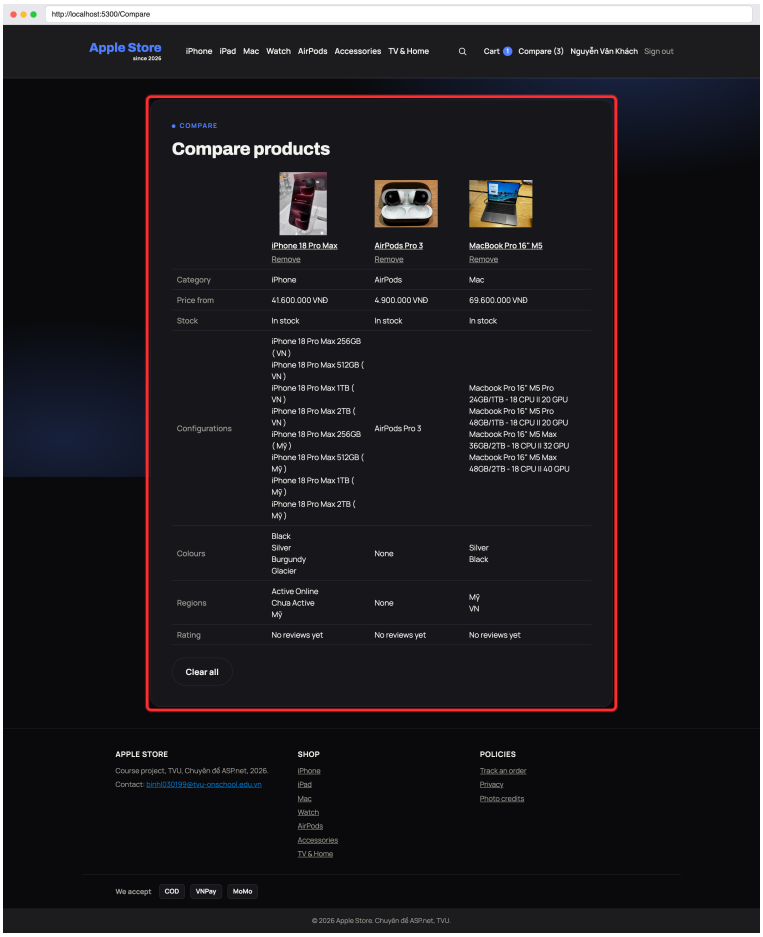
Hình 4.11: Trang nhập mã OTP gửi qua email



Hình 4.12: Trang đăng nhập (khung đỏ đánh dấu khối chính)

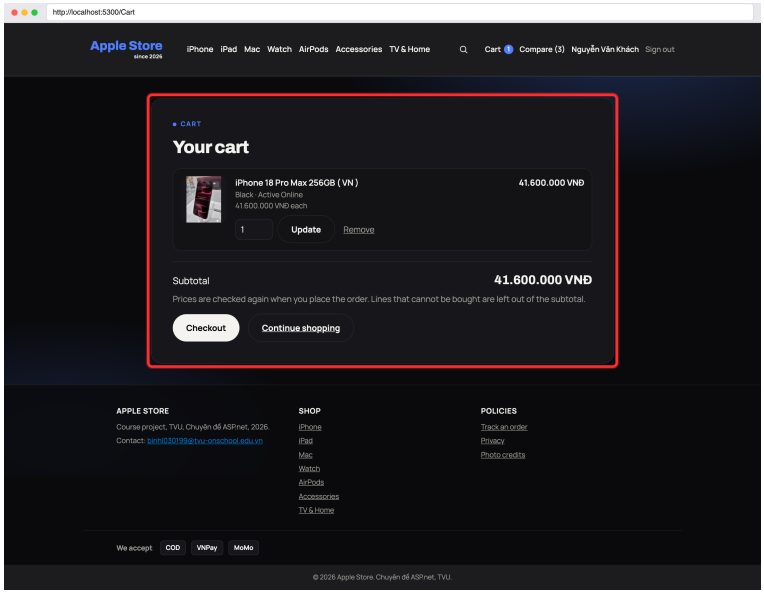
4.3.5. So sánh sản phẩm

Khách (kể cả chưa đăng nhập) chọn tối đa 3 sản phẩm thuộc bất kỳ danh mục nào để so sánh giá, tình trạng còn hàng, cấu hình, màu, khu vực và điểm đánh giá.

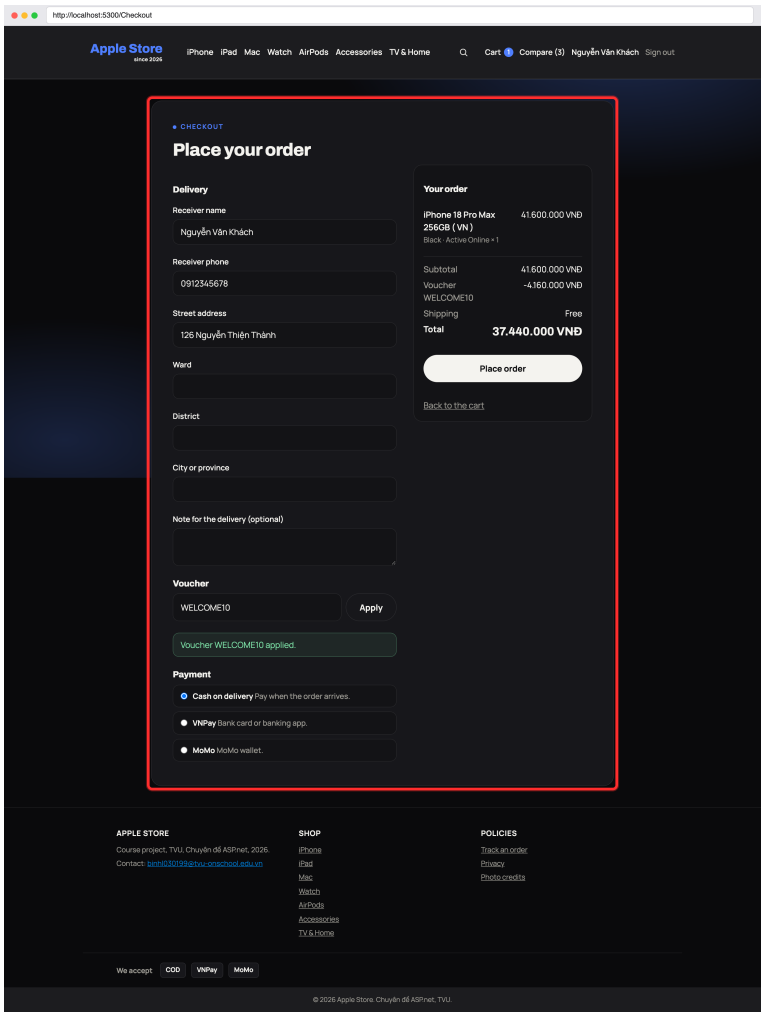


Hình 4.13: Trang so sánh sản phẩm (khung đỏ đánh dấu khối chính)

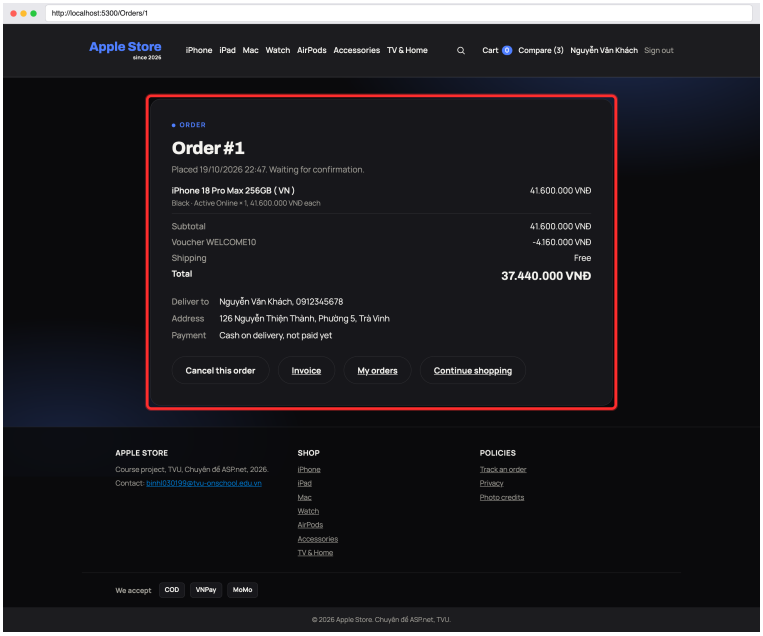
4.3.6. Giỏ hàng, thanh toán và đơn hàng



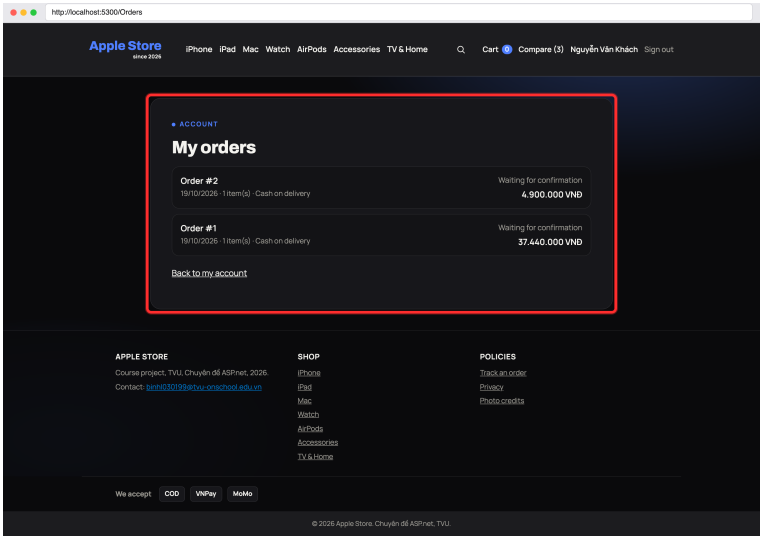
Hình 4.14: Giỏ hàng (khung đỏ đánh dấu khối chính)



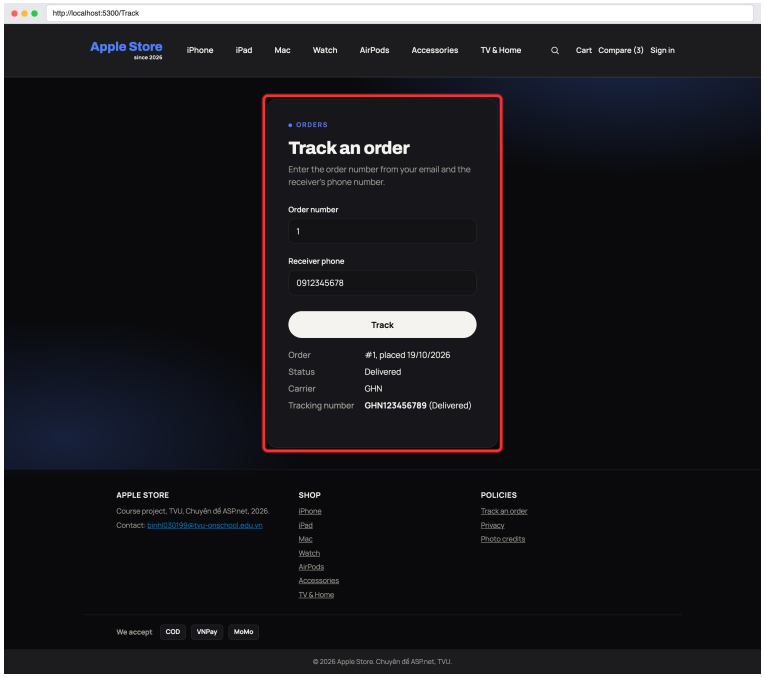
Hình 4.15: Trang thanh toán đã áp dụng voucher (khung đỏ đánh dấu khối chính)



Hình 4.16: Trang đơn hàng sau khi đặt (khung đỏ đánh dấu khối chính)



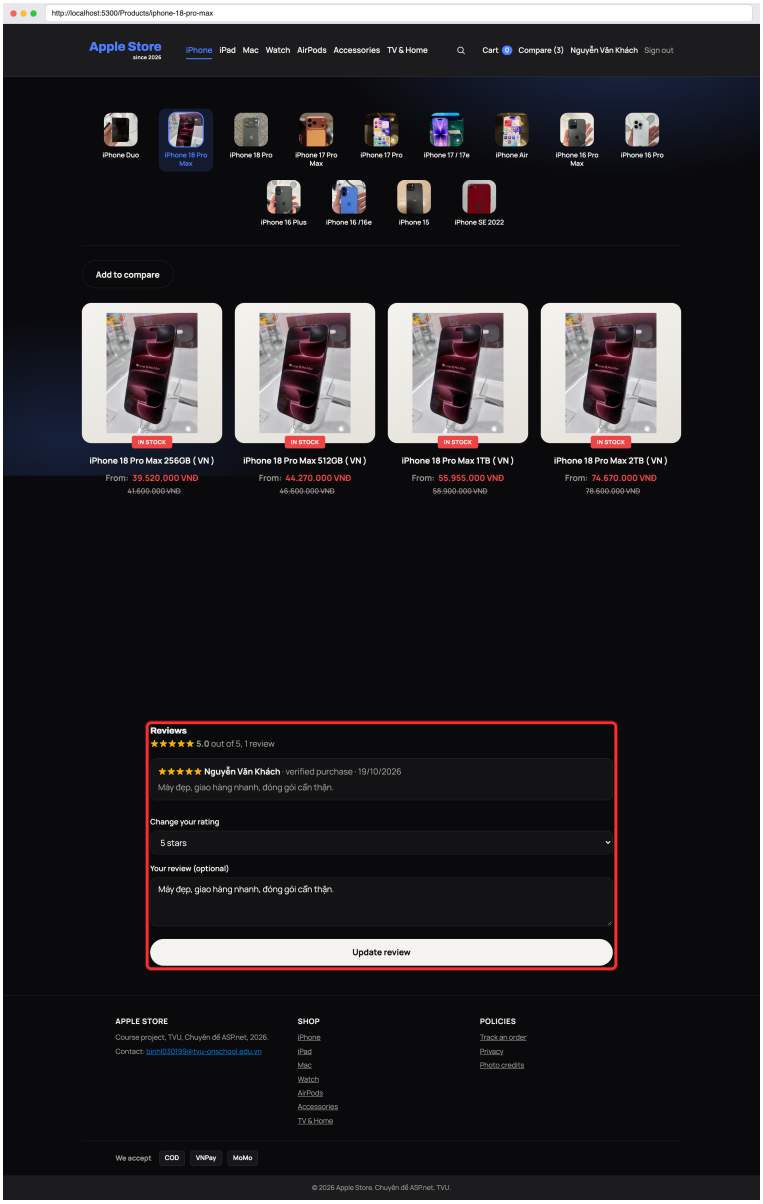
Hình 4.17: Danh sách đơn hàng của khách (khung đỏ đánh dấu khối chính)



Hình 4.18: Tra cứu đơn hàng bằng mã đơn và số điện thoại (khung đỏ đánh dấu khối chính)

4.3.7. Đánh giá sản phẩm

Chỉ khách đã nhận một đơn có sản phẩm đó mới được đánh giá; mỗi khách một đánh giá cho một sản phẩm và có thể sửa lại.

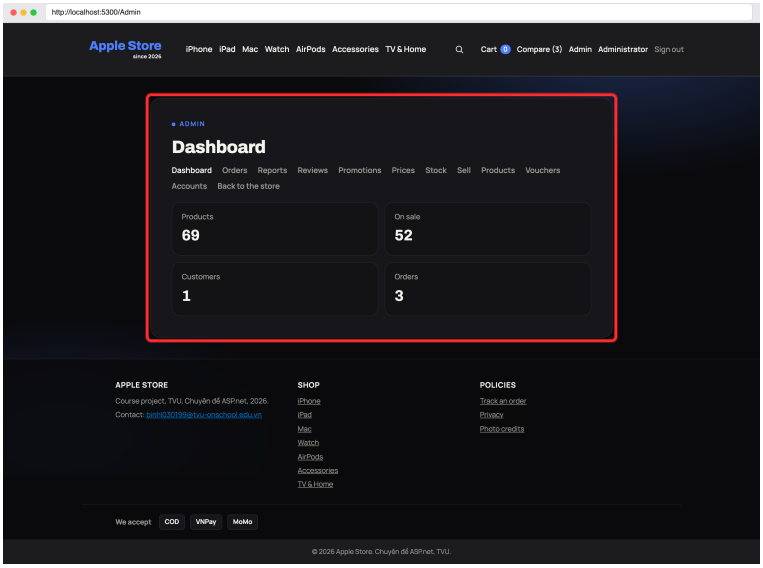


Hình 4.19: Khối đánh giá trên trang sản phẩm (khung đỏ đánh dấu khối chính)

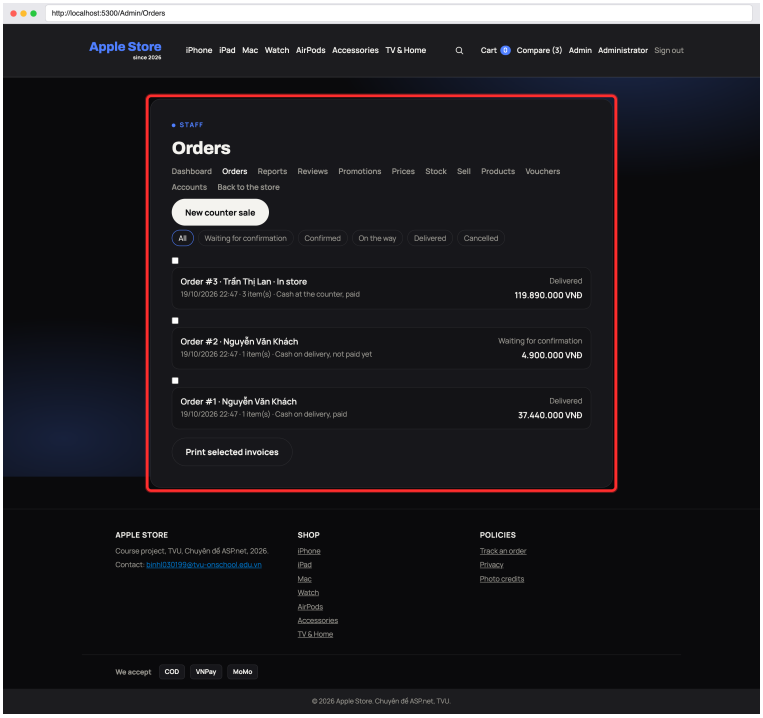
4.4. Giao diện trang quản trị

Trang quản trị nằm trong Area Admin. Quản trị viên thấy đủ các mục; nhân viên chỉ thấy Đơn hàng, Báo cáo, Đánh giá, Khuyến mãi, Giá, Kho và Bán hàng.

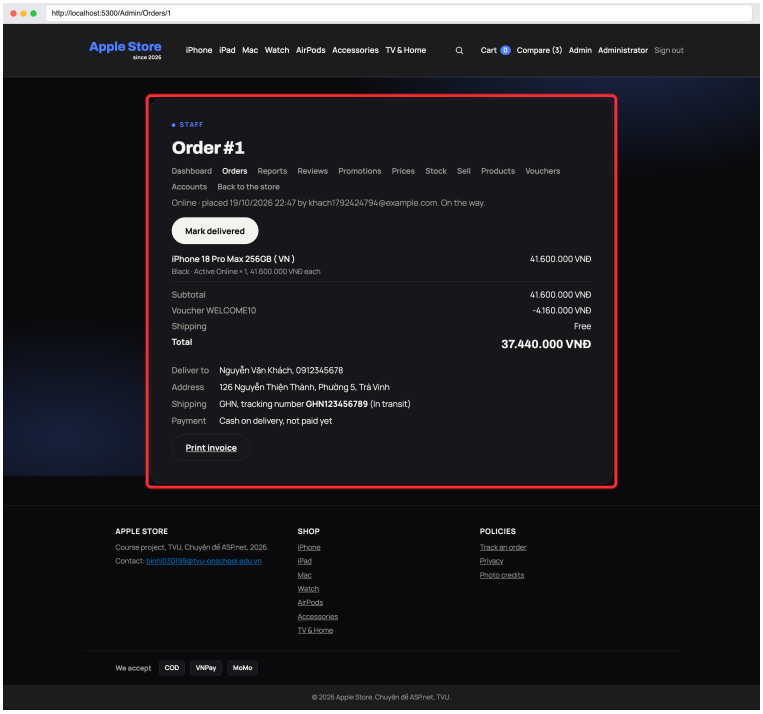
4.4.1. Tổng quan và xử lý đơn hàng



Hình 4.20: Trang tổng quan của quản trị viên (khung đỏ đánh dấu khối chính)



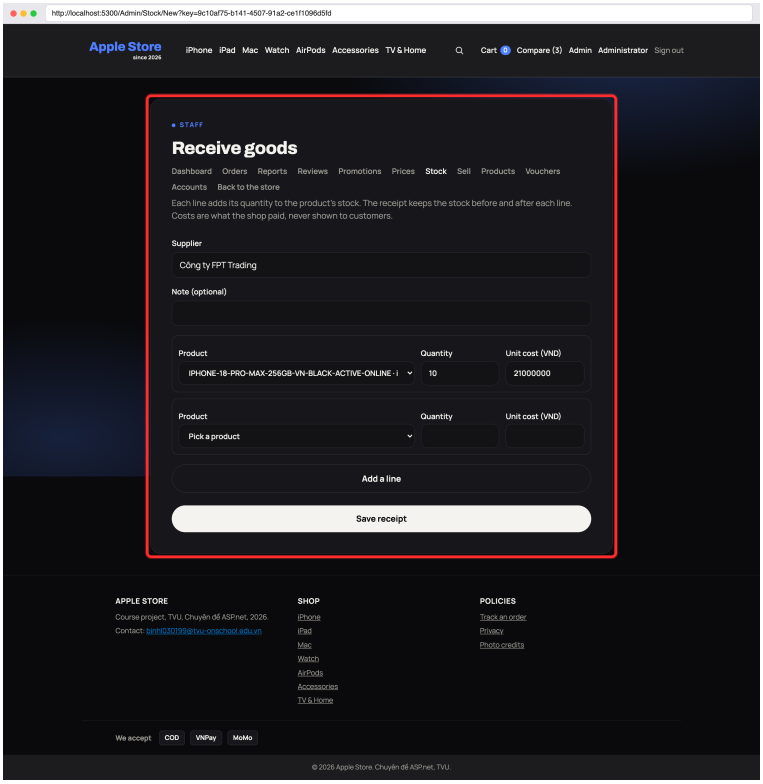
Hình 4.21: Danh sách đơn hàng theo trạng thái (khung đỏ đánh dấu khối chính)



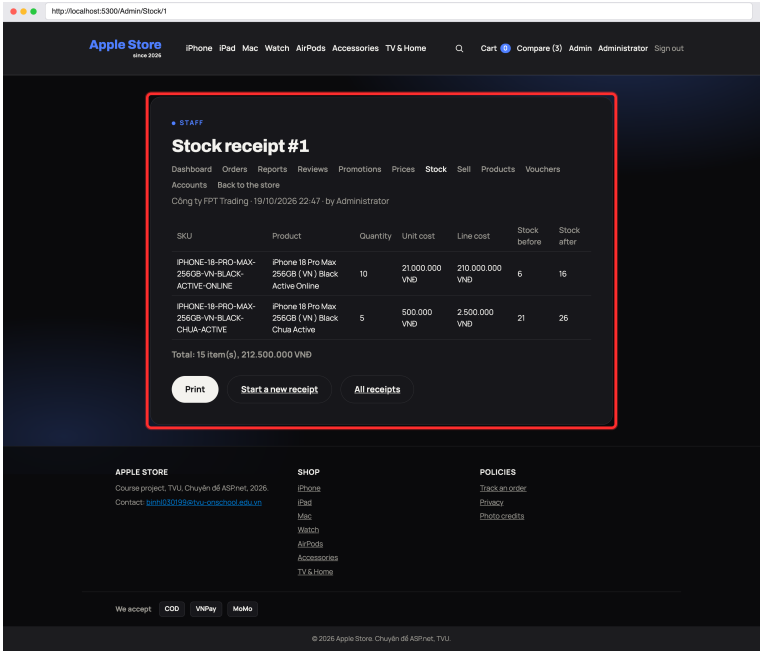
Hình 4.22: Chi tiết đơn hàng và các nút xử lý theo trạng thái (khung đỏ đánh dấu khối chính)

4.4.2. Nhập kho và tồn kho

Nhân viên lập phiếu nhập với nhà cung cấp, sản phẩm, số lượng và giá nhập. Trang phiếu cho thấy tồn trước và tồn sau của từng dòng, đúng công thức tồn cuối = tồn đầu + nhập. Trang tồn kho liệt kê mọi biến thể, tồn thấp nhất lên đầu, tìm được theo SKU hoặc tên.



Hình 4.23: Lập phiếu nhập kho (khung đỏ đánh dấu khối chính)



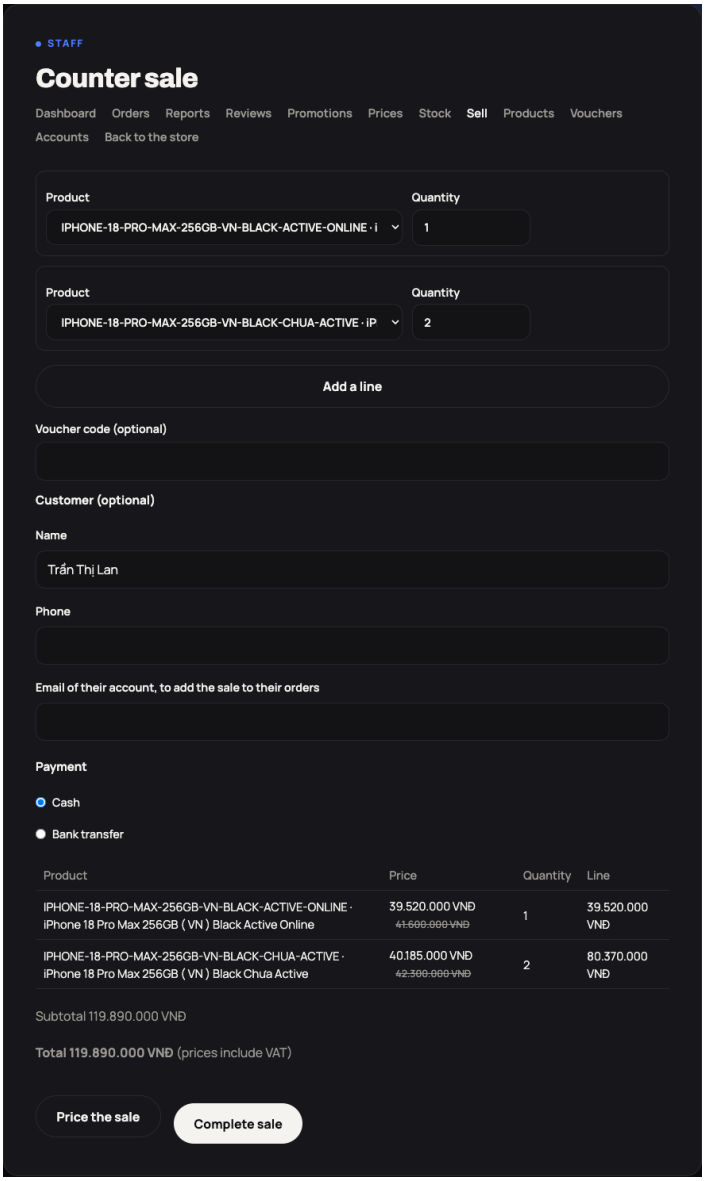
Hình 4.24: Phiếu nhập đã lưu: tồn trước và tồn sau của từng dòng (khung đỏ đánh dấu khối chính)



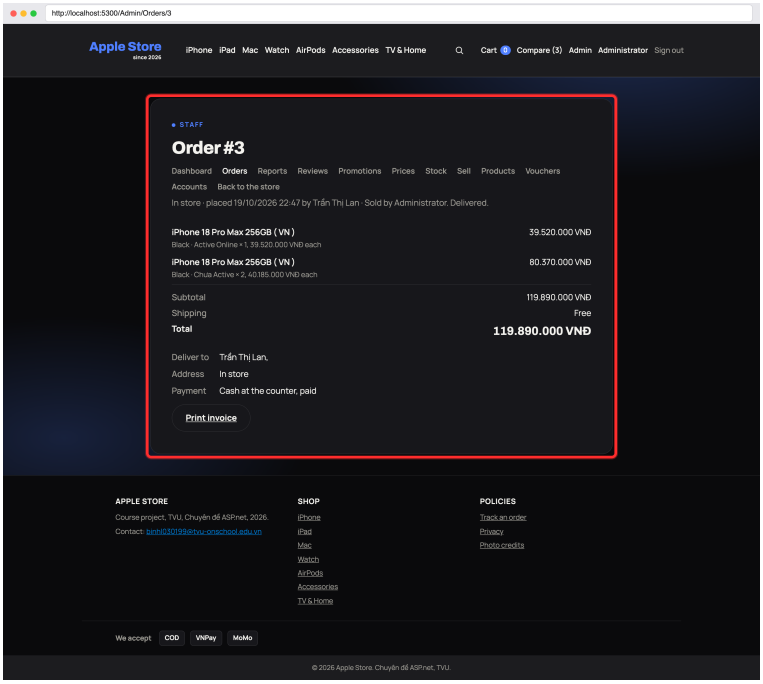
Hình 4.25: Danh sách tồn kho (khung đỏ đánh dấu khối chính)

4.4.3. Bán hàng tại quầy và in hóa đơn hàng loạt

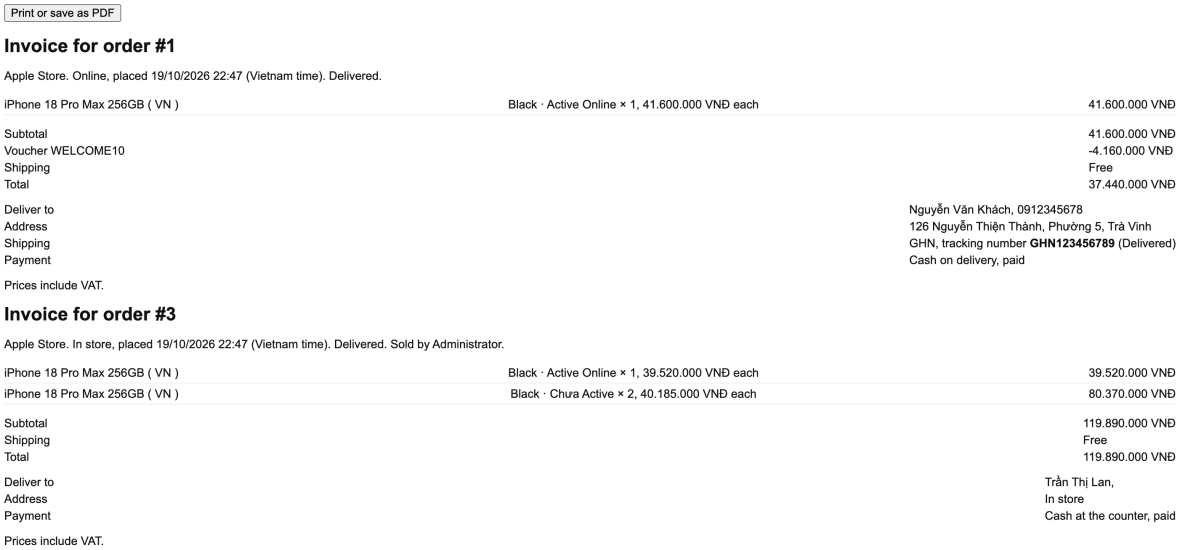
Nhân viên chọn sản phẩm, bấm Tính tiền để xem giá đang bán (có khuyến mãi và voucher), rồi bấm Hoàn tất bán. Lần bán trở thành một đơn hàng loại Tại quầy, đã thanh toán và hoàn tất. Từ danh sách đơn, nhân viên đánh dấu nhiều đơn và in tất cả hóa đơn cùng lúc.



Hình 4.26: Bán hàng tại quầy: bảng tính tiền trước khi hoàn tất

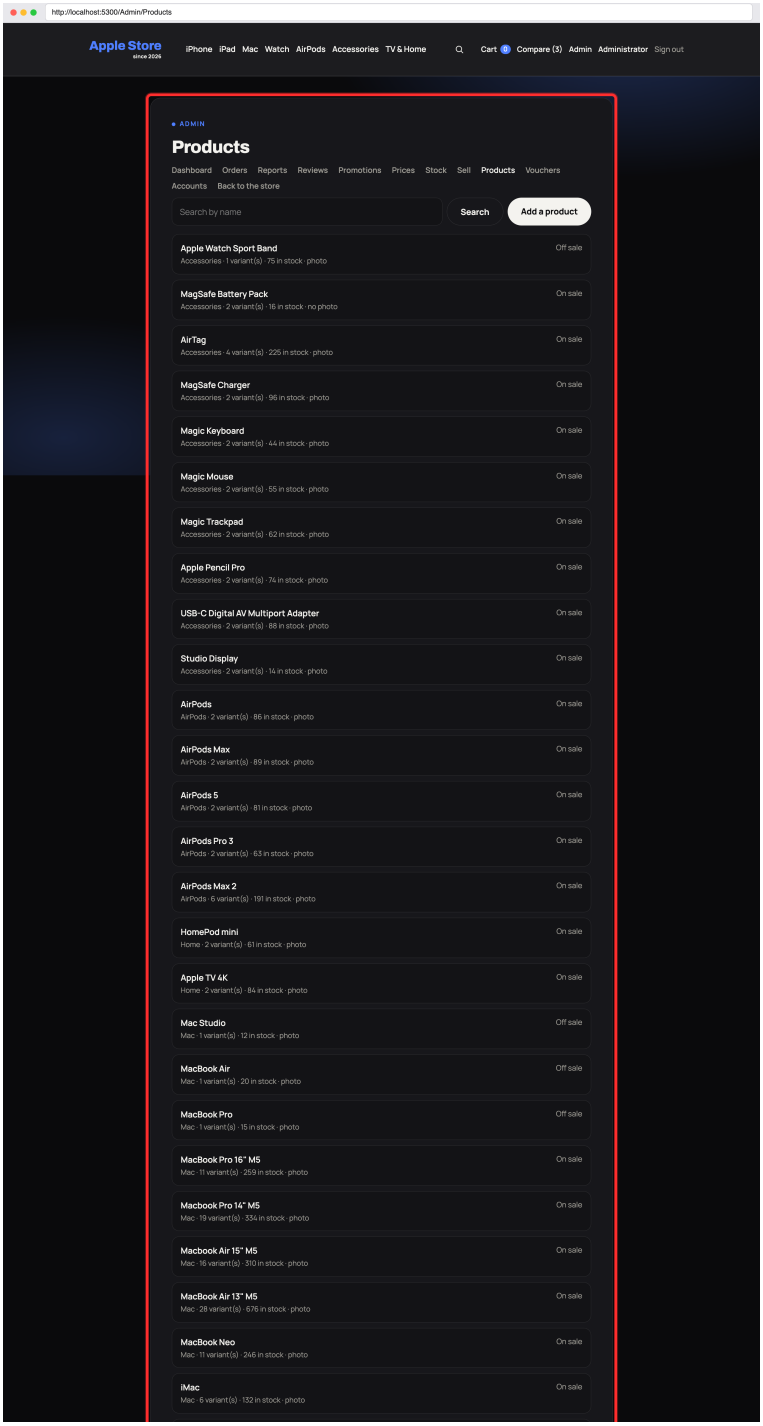


Hình 4.27: Đơn bán tại quầy: kênh Tại quầy và nhân viên bán (khung đỏ đánh dấu khối chính)

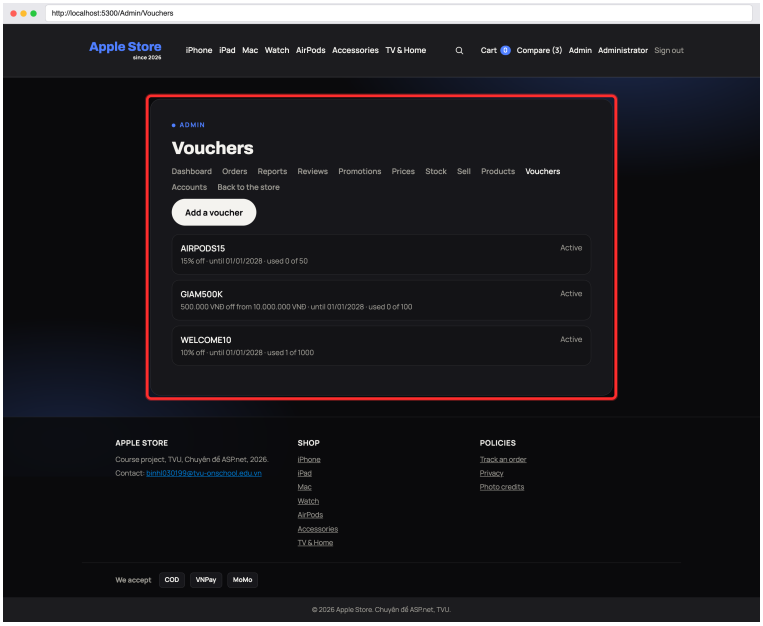


Hình 4.28: In nhiều hóa đơn cùng lúc

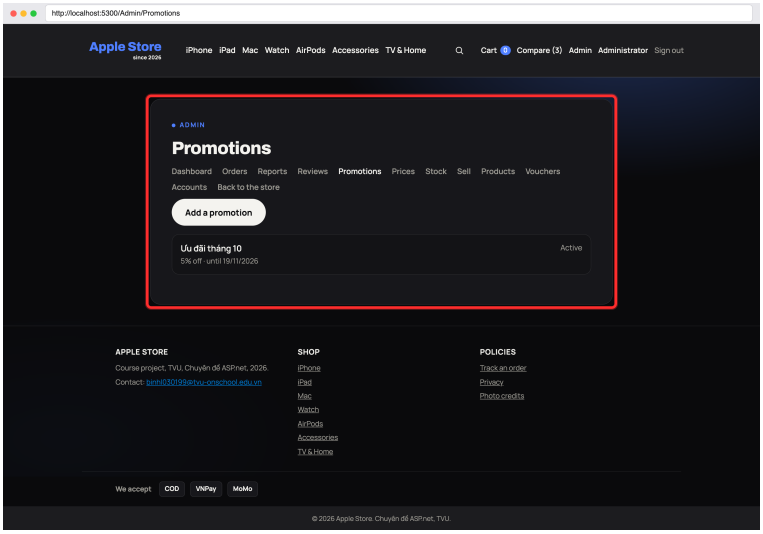
4.4.4. Sản phẩm, voucher, khuyến mãi và giá



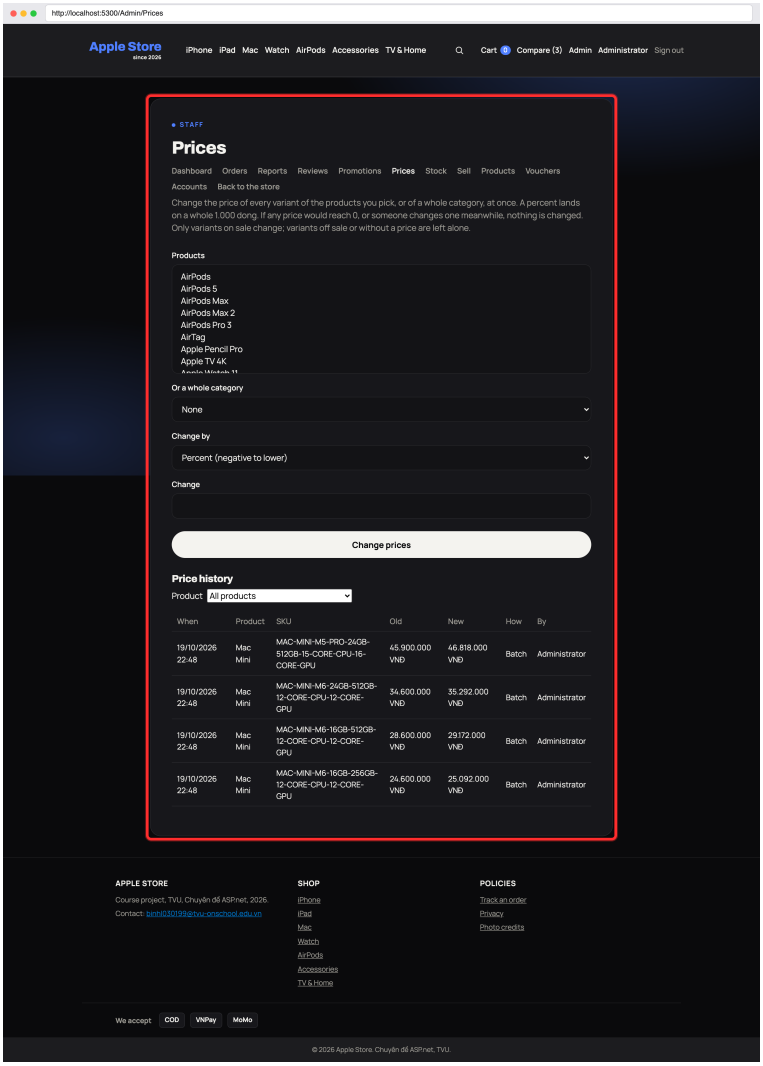
Hình 4.29: Quản lý sản phẩm (khung đỏ đánh dấu khối chính)



Hình 4.30: Quản lý voucher (khung đỏ đánh dấu khối chính)

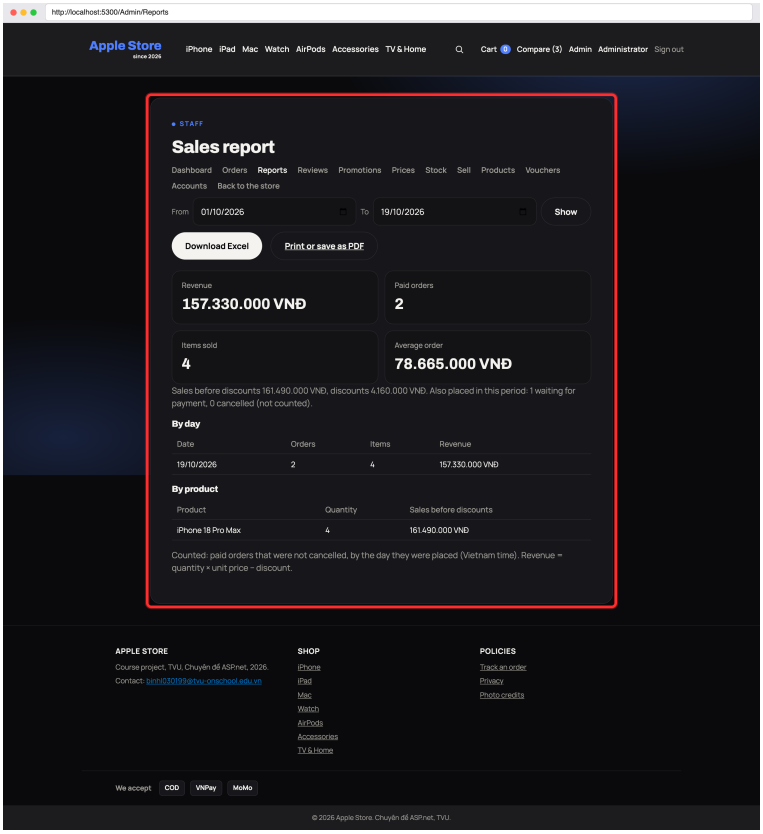


Hình 4.31: Quản lý khuyến mãi (khung đỏ đánh dấu khối chính)

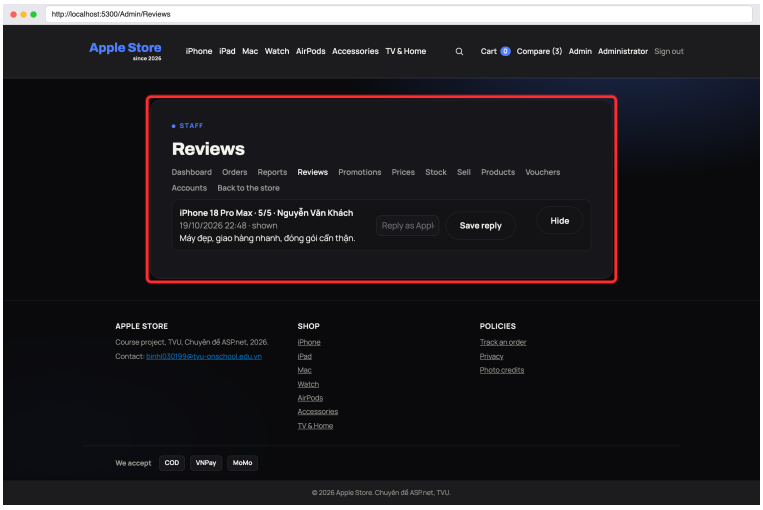


Hình 4.32: Đổi giá hàng loạt và lịch sử đổi giá (khung đỏ đánh dấu khối chính)

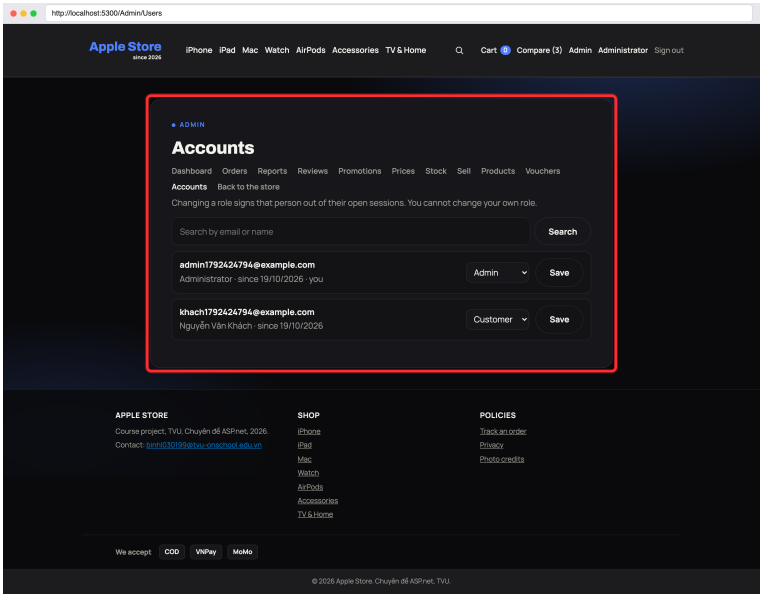
4.4.5. Báo cáo, đánh giá và tài khoản



Hình 4.33: Báo cáo doanh thu, xuất Excel và PDF (khung đỏ đánh dấu khối chính)



Hình 4.34: Quản lý đánh giá: trả lời và ẩn (khung đỏ đánh dấu khối chính)



Hình 4.35: Quản lý tài khoản và vai trò (khung đỏ đánh dấu khối chính)

4.5. Kiểm thử và đánh giá

4.5.1. Kiểm thử tự động

Bộ test hiện có 647 test xUnit, tất cả đều đạt. Trước mỗi lần gộp mã, cổng kiểm tra gồm: dotnet build --no-incremental (0 cảnh báo, 0 lỗi), dotnet test, dotnet format --verify-no-changes và kịch bản kiểm tra trên trình duyệt của tính năng đó.

4.5.2. Kiểm thử trên trình duyệt

Bảng 4.3: Các kịch bản kiểm tra trên trình duyệt (chạy lại ngày 12/10/2026)

Kịch bản	Nội dung	Số mục kiểm tra
verify-account	Đăng ký, OTP, đăng nhập, khóa tài khoản, quên và đổi mật khẩu, hồ sơ, địa chỉ	69
verify-admin	Khu vực quản trị, phân quyền, tài khoản quản trị đầu tiên	31
verify-cart	Giỏ hàng	23
verify-checkout	Đặt hàng và voucher	25
verify-payment	Thanh toán VNPay, MoMo giả lập	16
verify-orders	Đơn hàng, tra cứu và khóa tra cứu, xử lý đơn, email	21
verify-admin-catalog	Quản trị sản phẩm, voucher, vai trò, số tiền nguyên đồng	20
verify-reports	Báo cáo, file Excel, PDF, hóa đơn	16

Kịch bản	Nội dung	Số mục kiểm tra
verify-reviews	Đánh giá và trả lời	9
verify-compare	So sánh sản phẩm	12
verify-catalog-filter	Lọc theo giá, sắp xếp mới nhất	16
verify-promotions	Khuyến mãi	15
verify-prices	Đổi giá hàng loạt và lịch sử giá	12
verify-stock	Nhập kho, phiếu nhập, tồn kho	13
verify-sales	Bán hàng tại quầy, in hóa đơn hàng loạt	11
verify-responsive	Mọi trang ở 5 chiều rộng màn hình, không trang nào cuộn ngang	128
Tổng	16 kịch bản	437

4.5.3. Kiểm thử đột biến và rà soát mã

Sau mỗi tính năng, các dòng mã quyết định được cố ý làm sai để chắc có test phát hiện, ví dụ bỏ điều kiện tồn kho, bỏ kiểm tra chữ ký thanh toán, cho phép giá về 0. Một số lần có đột biến không bị phát hiện; khi đó hoặc viết thêm test chặt hơn, hoặc nhận ra dòng mã đó là thừa và bỏ đi. Mỗi pull request còn được rà soát bởi một lượt review độc lập, tìm các tình huống chạy song song, ghi dữ liệu chưa kiểm soát và phân quyền.

4.5.4. Một số lỗi phát hiện và đã khắc phục

Bảng 4.4: Một số lỗi phát hiện trong quá trình kiểm thử

Lỗi	Phát hiện bởi	Cách khắc phục
Các form quản trị viết đường dẫn tay không có mã chống giả mạo (trình duyệt nhận lỗi 400)	Kiểm tra trên trình duyệt	Thêm mã; thêm test đi qua mọi trang kiểm tra mọi form
Giá nhập dạng "24.990.000" bị lưu thành rỗng	Rà soát toàn bộ ứng dụng	Kiểm tra ModelState và báo lỗi trên form
Mã OTP đăng ký đoán được không giới hạn	Rà soát toàn bộ ứng dụng	Hủy lần đăng ký sau 5 lần sai
Đơn đã hủy vẫn thanh toán trực tuyến được	Test thanh toán lại	Từ chối thanh toán; tiền đến muộn được ghi nhận, đơn vẫn hủy
Giỏ hàng in ra dòng mã Razor bên cạnh giá	Kiểm tra trên trình duyệt	Sửa cách viết; thêm test không cho mã lọt ra trang
"1.000" nhập ở ô đổi giá bị hiểu là	Test giao diện	Số tiền phải là số nguyên không

Lỗi	Phát hiện bởi	Cách khắc phục
1 đồng		dấu phân cách
Số tiền trên thẻ báo cáo bị tràn khỏi khung	Ảnh chụp cho báo cáo này	Sửa CSS; thêm phép đo vào kịch bản kiểm tra
Lưu phiếu nhập xong, bấm Quay lại của trình duyệt rồi lưu lần nữa thì hàng được nhập hai lần (tồn 60 thành 70)	Kịch bản kiểm tra nhập kho	Đưa mã form vào địa chỉ trang; mã đã dùng thì chuyển tới phiếu đã lưu
Trang chi tiết và hóa đơn của đơn khách lẻ (không có tài khoản) bị lỗi	Test giao diện bán tại quầy	Đọc email khách theo cách cho phép không có tài khoản
Tên biến thể bị lặp ("iPhone 18 Pro Max iPhone 18 Pro Max 256GB") trên trang kho và bán hàng	Ảnh chụp cho báo cáo này	Một hàm duy nhất VariantText đặt tên biến thể, có test
Mọi trang khi đã đăng nhập bị cuộn ngang ở chiều rộng 992 đến 1061 điểm ảnh: thanh menu mở rộng quá sớm	Quét giao diện ở 5 chiều rộng	Cho menu mở rộng từ 1200 điểm ảnh; thêm kịch bản verify-responsive
Bảng ghi nguồn ảnh (trang Credits) tràn ngang trên điện thoại	Rà lại các trang chưa kịch bản nào mở	Cho riêng bảng cuộn ngang trong khung của nó
Ô giá sản phẩm, giá biến thể và voucher vẫn nhận "1.000" thành 1 đồng	Rà soát các ô số tiền	Một thuộc tính kiểm tra dùng chung WholeDongAttribute cho mọi ô số tiền
Đổi giá cả danh mục Watch thêm 2% bị từ chối: biến thể cũ đã ngừng bán giá 399 đồng, làm tròn đến 1.000 thành 0	Ảnh chụp cho báo cáo này	Chỉ đổi giá biến thể đang bán (chủ dự án quyết định); thêm test và kiểm tra trên danh mục thật

4.5.5. Quy trình quản lý mã nguồn

Mỗi tính năng là một nhánh riêng (ví dụ feat/cart, feat/promotions, feat/price-history) và được gộp vào nhánh dev qua pull request sau khi qua đủ cổng kiểm tra. Từ PR #12 đến PR #40, mỗi pull request ghi rõ quyết định nào của chủ dự án, quyết định nào của người làm, và kết quả kiểm tra. Tiến độ được ghi hằng tuần trong thư mục progress-report/ và nhật ký kiểm tra trong docs/verification.md.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

5.1.1. Kết quả đạt được

- Hoàn thành đủ 37 use case của đặc tả: tài khoản với OTP qua email thật, danh mục với tìm kiếm, lọc theo giá và so sánh, giỏ hàng, đặt hàng với voucher, thanh toán COD và VNPay, MoMo giả lập, xử lý đơn hàng với email thông báo, nhập kho và kiểm tra tồn kho, bán hàng tại quầy, quản trị sản phẩm, voucher, khuyến mãi, giá và lịch sử giá, tài khoản và vai trò, đánh giá, báo cáo doanh thu xuất Excel và PDF, in hóa đơn (kể cả in nhiều hóa đơn cùng lúc).
- Sử dụng đúng các cơ chế của ASP.NET Core: MVC và Razor, Tag Helper, View Component, Area, Dependency Injection, Options pattern, model validation, anti-forgery, EF Core Code First với 24 migration, ASP.NET Core Identity trên bảng Users của đặc tả.
- Dữ liệu đúng khi nhiều người thao tác cùng lúc nhờ giao dịch và cập nhật có điều kiện; các tình huống này đều có test.
- 647 test tự động và 16 kịch bản kiểm tra trên trình duyệt (437 mục, trong đó 128 mục kiểm tra năm chiều rộng màn hình), tất cả đều đạt.

5.1.2. Ý nghĩa của đề tài

Đề tài giúp em hiểu cách một ứng dụng ASP.NET Core được tổ chức từ cơ sở dữ liệu tới giao diện, và cách dùng các cơ chế có sẵn thay vì tự viết lại. Việc viết test trước và kiểm tra trên trình duyệt thật cho thấy nhiều lỗi chỉ lộ ra khi chạy ứng dụng thật hoặc khi hai yêu cầu đến cùng lúc.

5.1.3. Hạn chế

- VNPay và MoMo mới được giả lập trong ứng dụng, chưa kết nối sandbox thật.
- Chưa giới hạn số lần yêu cầu gửi mã OTP (đăng ký, quên mật khẩu); việc đoán mã đã bị giới hạn 5 lần sai.
- Mã SKU chưa có ràng buộc duy nhất.
- Khi trang tra cứu đơn báo khóa, người thử biết mã đơn đó có thật; đây là đánh đổi giống thông báo khóa tài khoản của Identity.
- Một số trang làm từ đầu (địa chỉ, hồ sơ, mở trang thanh toán) khi cơ sở dữ liệu lỗi thì hiện trang lỗi chung thay vì thông báo riêng.

- Đơn trực tuyến chưa thanh toán vẫn giữ tồn kho cho tới khi bị hủy.
- Bảng thuộc tính sản phẩm còn trống nên trang so sánh dùng giá, cấu hình, màu, khu vực và đánh giá; chức năng yêu thích (bảng Favorites) chưa làm.
- Ứng dụng chưa được triển khai lên máy chủ; cơ sở dữ liệu hiện là SQLite, chưa chạy thử trên SQL Server.

5.2. Hướng phát triển

- Kết nối VNPay và MoMo sandbox thật khi có tài khoản.
- Bật bộ giới hạn tần suất có sẵn của ASP.NET Core (rate limiter) cho các form gửi mã OTP.
- Thêm báo cáo nhập xuất theo kỳ (BM_REPORT_INOUT_01) từ dữ liệu phiếu nhập và đơn bán đã có.
- Chuyển sang SQL Server và triển khai bằng Docker hoặc dịch vụ đám mây, có HTTPS.
- Thêm thuộc tính sản phẩm theo danh mục để so sánh chi tiết hơn, chức năng yêu thích và thông báo qua SMS.
- Tự động hủy đơn trực tuyến quá hạn thanh toán để trả lại tồn kho.

TÀI LIỆU THAM KHẢO

[1] Khoa Công nghệ Thông tin, Trường Đại học Trà Vinh (2025), Đồ án môn học Công nghệ phần mềm: Xây dựng website Apple Store (đặc tả yêu cầu, bản tóm tắt trong docs/requirements.md của kho mã nguồn).

[2] Microsoft, ASP.NET Core documentation, https://learn.microsoft.com/aspnet/core/

[3] Microsoft, Entity Framework Core documentation, https://learn.microsoft.com/ef/core/

[4] Microsoft, .NET documentation, https://learn.microsoft.com/dotnet/

[5] xUnit.net, https://xunit.net/

[6] Microsoft Playwright, https://playwright.dev/

[7] ClosedXML, https://github.com/ClosedXML/ClosedXML

[8] MailKit, https://github.com/jstedfast/MailKit

[9] VNPay, Tài liệu tích hợp cổng thanh toán, https://sandbox.vnpayment.vn/apis/

[10] Bootstrap, https://getbootstrap.com/